



СПбГЭТУ «ЛЭТИ»
ПЕРВЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ

Нейронные сети



Speaker

Date

| www.etu.ru

Лабораторная работа 1

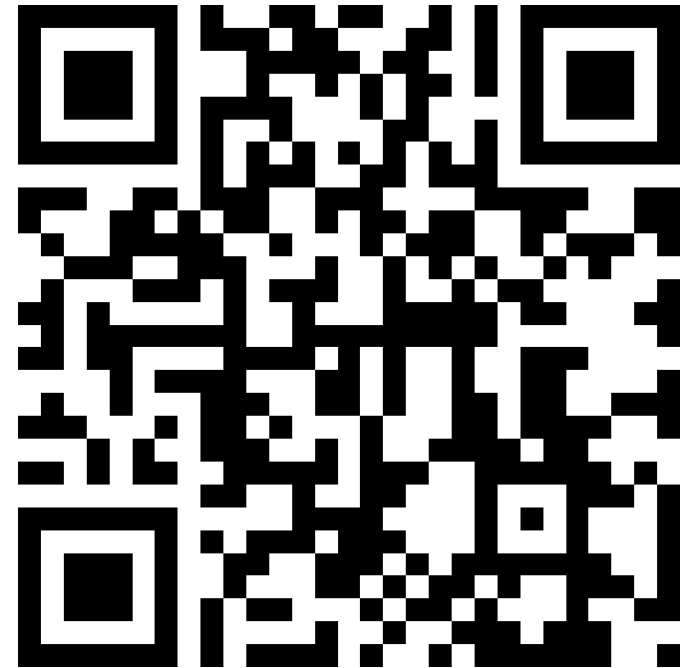
Сверточные сети

Важные ссылки

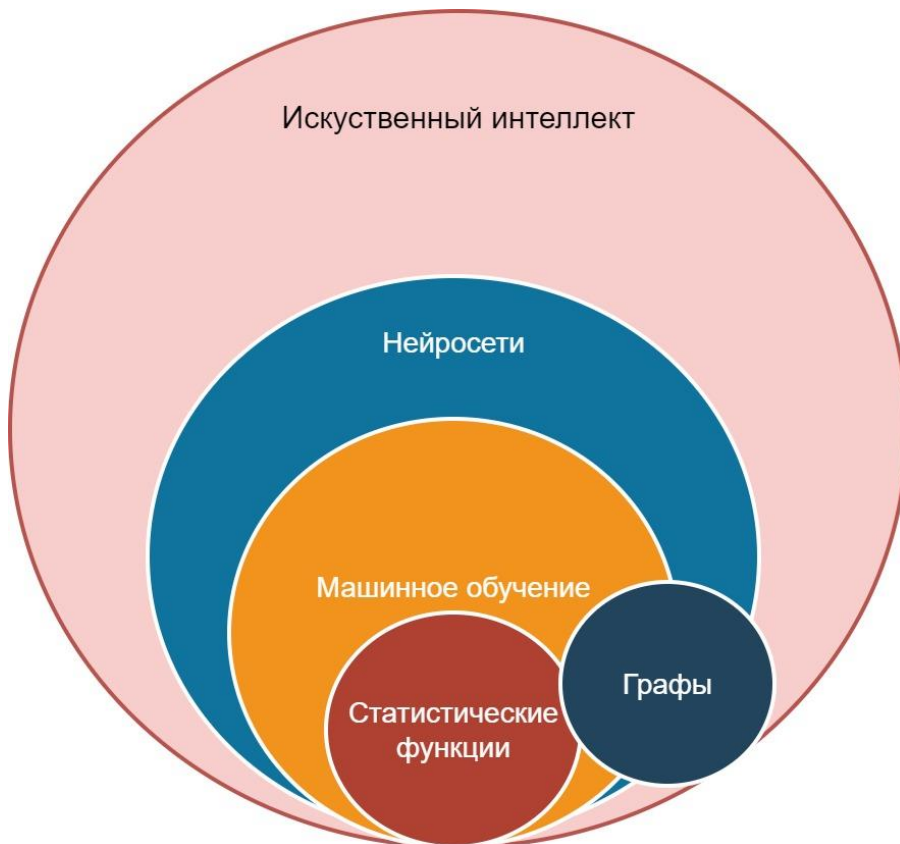
Папка с материалами,
заданиями и для работ



Папка для загрузки
лабораторных



Машинное обучение, нейросети, графы, ИИ



- **Статистические функции** — математические инструменты, позволяющие анализировать, интерпретировать и делать выводы на основе наборов данных
- **Машинное обучение** – применение статистических функций, позволяющих сначала их «обучать» на наборе данных, а потом применить эти алгоритмы на аналогичных
- **Нейросети** - специфическая архитектура алгоритмов машинного обучения. Состоят из слоев взаимосвязанных нейронов, каждый из которых обрабатывает определенный аспект входных данных.
- **Анализ графов** - набор методов, направленных на изучение связей между сущностями для выявления неочевидных зависимостей.
- **Искусственный интеллект** – совокупность всех алгоритмов, которые позволяют обрабатывать большие объёмы данных, предсказывать события или принимать решения.

История ИИ

1956

Впервые
использован
термин ИИ

1960-
1970

Программы,
решающие
задачи с
помощью
логики и
статистики

1980-
1990

Алгоритмы
обучения с
подкреплен
ием и
обратной
связи

2000-
е

Появление
нейронных
сетей и
данных для
их
обучения

2010-
е

Сети для
обработки
изображений
и
естественного
языка

2023

Генеративные
сети и
большие
языковые
модели

Нейронные сети.

Что это?

- Нейронные сети представляют собой семейство алгоритмов машинного обучения, вдохновленных структурой биологического мозга.
- Ключевая особенность — способность к автоматическому обнаружению и моделированию сложных нелинейных взаимосвязей в данных.
- Базовый строительный блок нейросетей — искусственный нейрон, математическая функция, принимающая взвешенные входные сигналы, суммирующая их и применяющая нелинейное преобразование (функцию активации).
- Несмотря на простоту отдельного нейрона, сети из множества таких элементов, организованных в слои, способны моделировать невероятно сложные функции.

Нейросети. Виды

Вид нейронной сети	Что делает?	Как применяется
Сверточные нейронные сети (CNN)	Принимает входное изображение, присваивает важность (изучаемые веса и смещения) аспектам или объектам изображении, отличает одно от другого.	Один из лучших методов по распознаванию и классификации изображений и видео
Рекуррентные нейронные сети (RNN)	Принимает данные, использует текущее входное значение и сохраненное в памяти значение для прогнозирования следующей последовательности	Распознавание речи. Обработка языка. Прогнозирование продаж, спроса, отклика от рекламы
Генеративно-состязательные сети (GAN)	Генератор принимает в качестве входных данных случайное распределение и изучает функцию отображения, которая отображает входные данные в желаемый результат, который является фактическим базовым распределением данных	Создание текстов, изображений и видео

Нейросети.

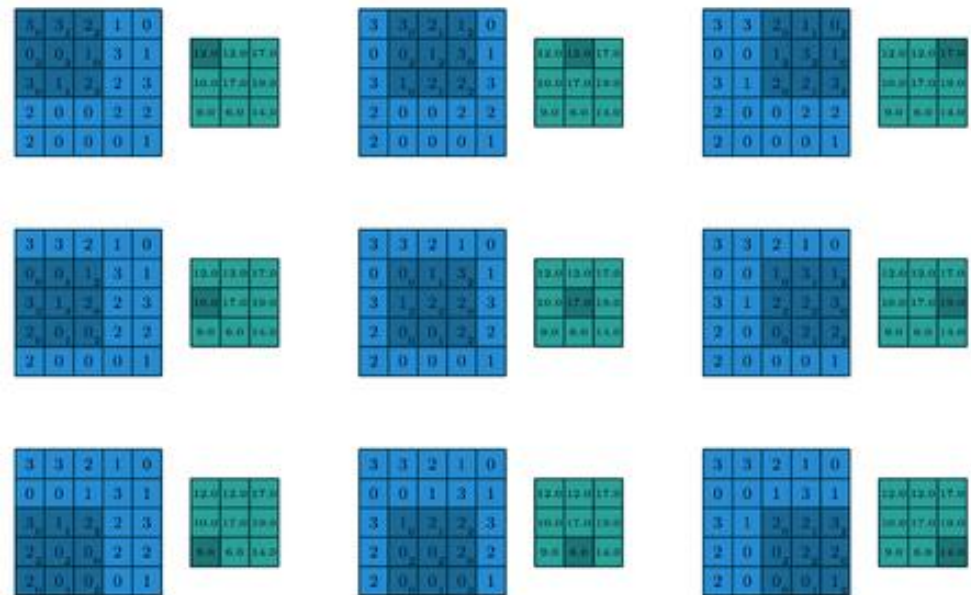
Преимущества и недостатки

- **Преимущества:**
 - Автоматическое извлечение признаков — способность самостоятельно находить значимые паттерны в сырых данных
 - Масштабируемость — производительность растет с увеличением объема данных и сложности архитектуры
 - Адаптивность — возможность тонкой настройки и переноса обучения между задачами (transfer learning)
- **Недостатки:**
 - высокие требования к вычислительным ресурсам,
 - потребность в больших объемах данных, зависимость от качества данных
 - склонность к переобучению
 - низкая интерпретируемость (проблема "черного ящика")

Свертка

- **Свертка** (англ. *convolution*) — операция над парой матриц A (размера $n \times n$) и B (размера $m \times m$), результатом которой является матрица $C = A * B$ размера $(n - m + 1) \times (n - m + 1)$.
- Каждый элемент результата вычисляется как скалярное произведение матрицы B и некоторой подматрицы A такого же размера (подматрица определяется положением элемента в результате).
- То есть,

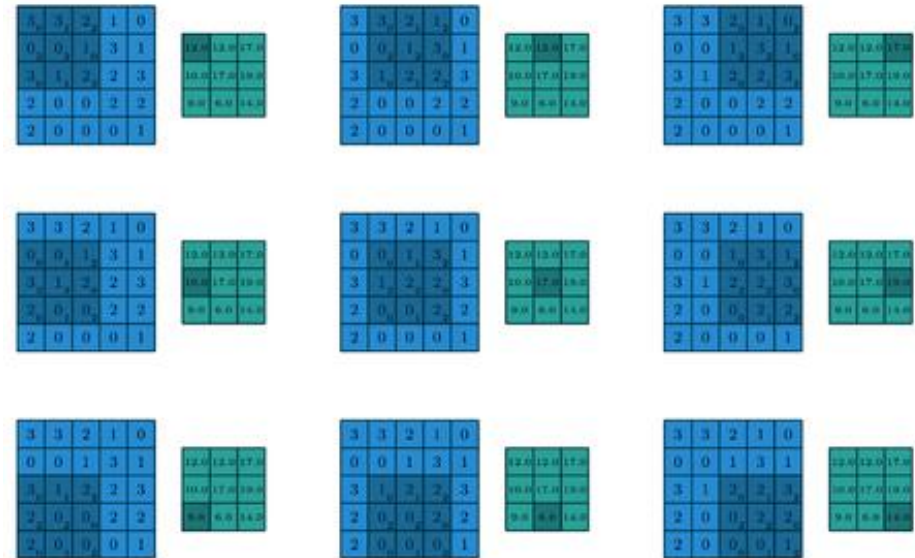
$$C_{i,j} = \sum_{u=0}^{m-1} \sum_{v=0}^{m-1} A_{i+u,j+v} B_{u,v}$$



Матрица B «двигается» по матрице A , и в каждом положении считается скалярное произведение матрицы B и той части матрицы A

Свертка

- Логический смысл свертки**
 — чем больше величина элемента свертки, тем больше эта часть матрицы A была похожа на матрицу B (похожа в смысле скалярного произведения).
- Поэтому матрицу A называют *изображением*, а матрицу B — *фильтром* или *образцом*.



Сверточная сеть

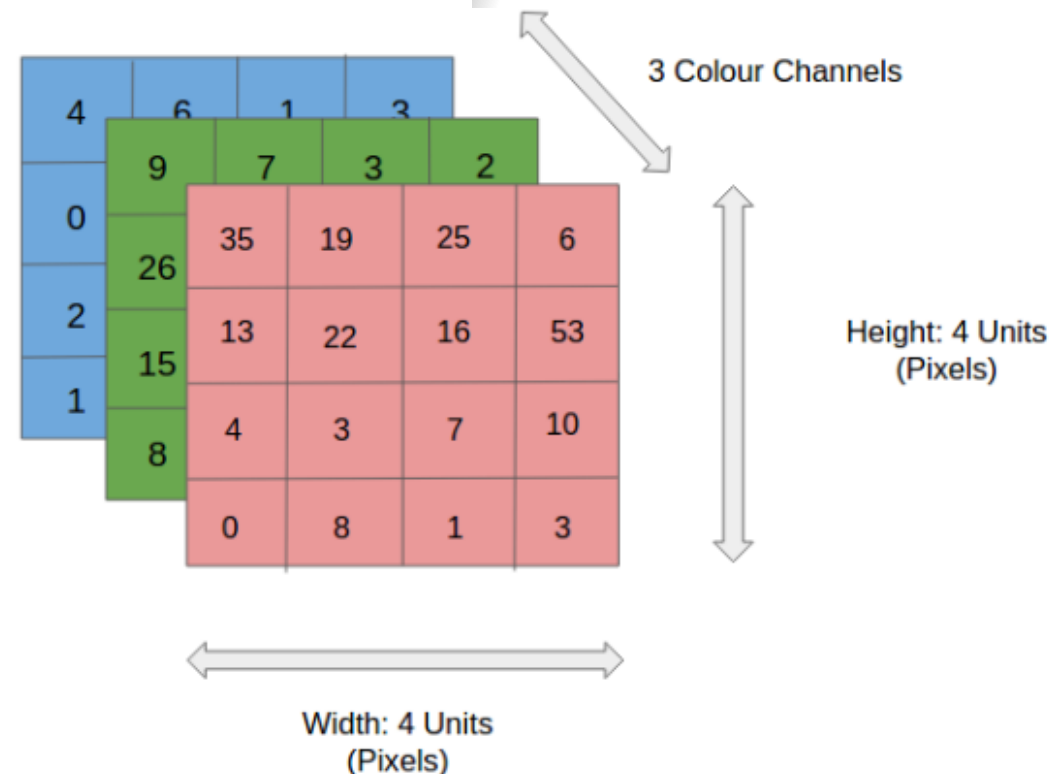
- **Свёрточная нейронная сеть (ConvNet/CNN)** — алгоритм глубокого обучения, который может:
 - принимать входное изображение
 - присваивать важность (изучаемые веса и смещения) аспектам или объектам изображению
 - отличать одно от другого.
- При этом изображения в сравнении с другими алгоритмами требуют гораздо меньше предварительной обработки.
- В примитивных методах фильтры разрабатываются вручную, но достаточно обученные сети CNN учатся применять эти фильтры/характеристики.

Сверточная сеть Преимущества.

- Один из лучших методов по распознаванию и классификации изображений (вероятность ошибки меньше, чем у человека)
- Меньшее количество настраиваемых весов по сравнению с полносвязной нейронной сетью
- Удобное распараллеливание вычислений (карты признаков обрабатываются независимо)
- Относительная устойчивость к повороту и сдвигу распознаваемого изображения
- Обучение при помощи классического метода обратного распространения ошибки

Структура сверточной сети

- В сверточной нейронной сети выходы промежуточных слоев образуют матрицу (изображение) или набор матриц (несколько слоёв изображения).
- На вход сверточной нейронной сети можно подавать три слоя изображения (R-, G-, B-каналы изображения).



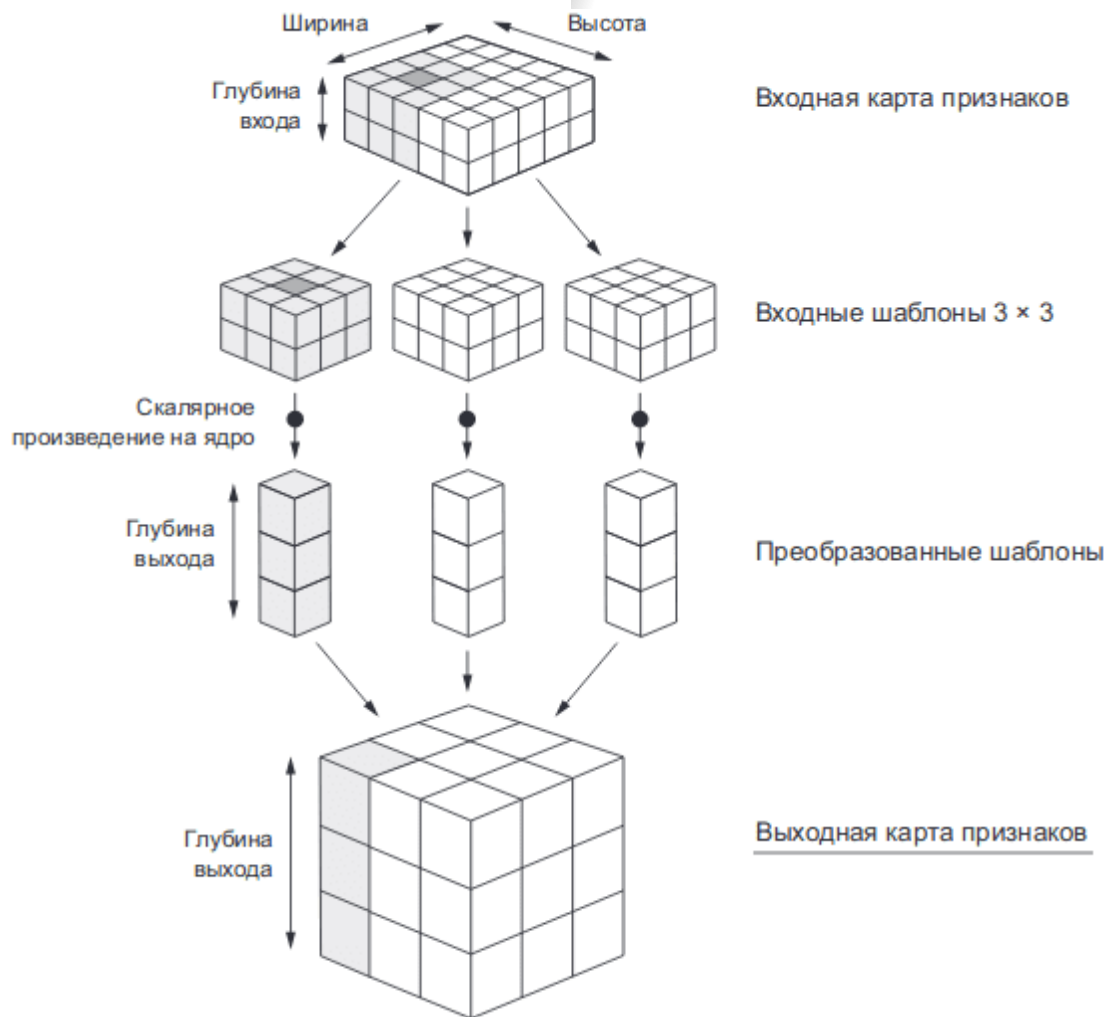
Изображение, разделённое на три цветовых плоскости (красную, зелёную и синюю) RGB-изображение, которое можно описать в разных цветовых пространствах — в оттенках серого (Grayscale), RGB, HSV, CMYK и т. д. ¹³

Структура сверточной сети

- В сверточной нейронной сети выходы промежуточных слоев образуют матрицу (изображение) или набор матриц (несколько слоёв изображения).
- На вход сверточной нейронной сети можно подавать три слоя изображения (R-, G-, B-каналы изображения).
- Основными видами слоев в сверточной нейронной сети являются
 - сверточные слои (англ. *convolutional layer*)
 - объединяющие (пулингговые) слои (англ. *pooling layer*)
 - слои активации
 - полносвязные слои (англ. *fully-connected layer*)

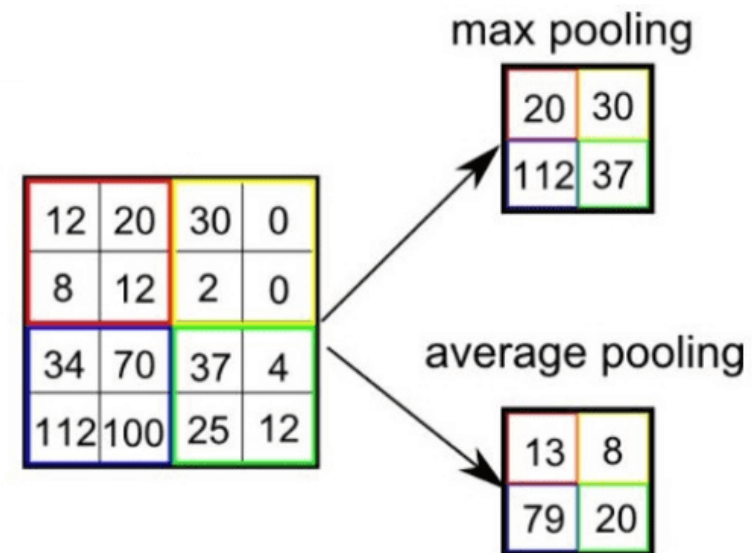
Сверточный слой

- Сверточный слой нейронной сети - применение операции свертки к выходам с предыдущего слоя, где веса ядра свертки являются обучаемыми параметрами.
- Еще один обучаемый вес используется в качестве константного сдвига (англ. bias).
- В одном сверточном слое может быть несколько сверток. В этом случае для каждой свертки на выходе получится своё изображение.



Объединяющий (пулинговый) слой

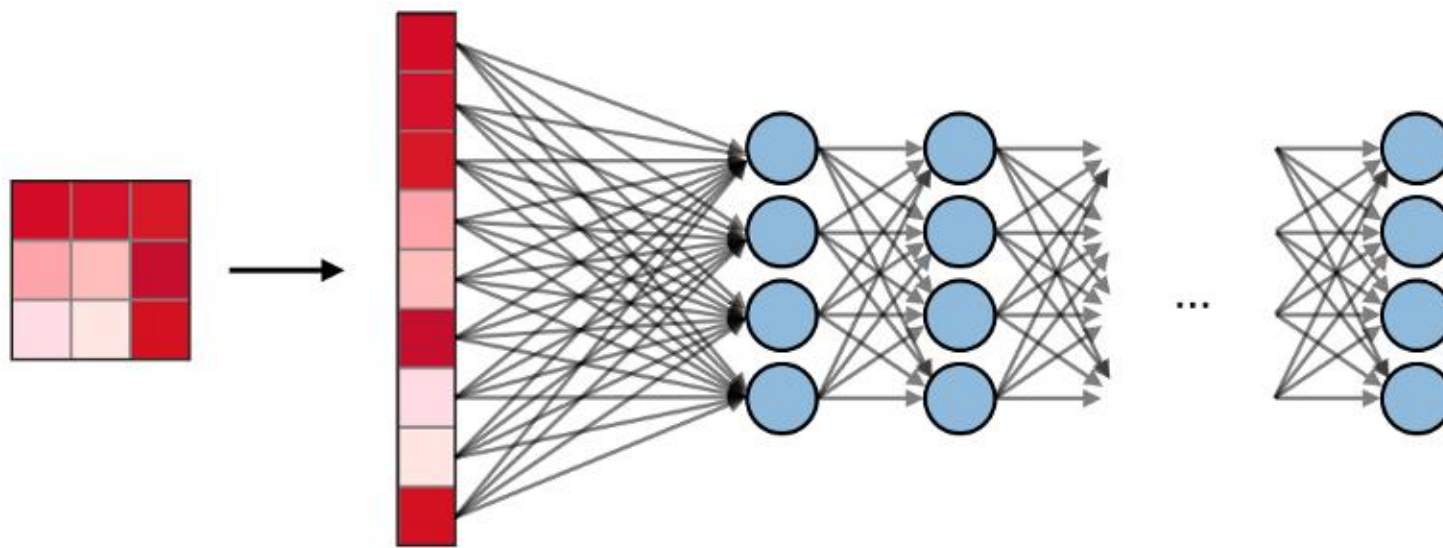
- Процесс сжатия (уменьшения размеров) изображения путём сложения значений блоков пикселей уменьшении размерности карты признаков.
- Нелинейное уплотнение карты признаков, при этом группа пикселей (обычно размера 2×2) уплотняется до одного пикселя, проходя нелинейное преобразование.
- Pooling имеет две разновидности: max-pooling и average-pooling.
- Скользящее окно $(2,2)$, двигается по карте признаков. Из выбранного шаблона выбирается максимальное (max-pooling) или среднее (average-pooling) значение. Формируется уменьшенная в размере карта признаков



Из матрицы $(4,4)$ получается выходная карта $(2,2)$ после операции max-pooling и average-pooling.

Полносвязный слой

- Работает с плоским входом, где каждый вход связан со всеми нейронами.
- Обычно они используются в конце сети для подключения скрытых слоев к выходному слою, что помогает оптимизировать оценки классов.
- Подсчитает оценки по классам, результирующая размерность будет равна $[1 \times 1 \times 10]$, где каждое из 10 значений будет соответствовать оценке определенного класса среди 10 категорий изображений из CIFAR-10. Как и в обычных нейронных сетях, каждый нейрон этого слоя будет связан со всеми нейронами предыдущего слоя (3D-представления)

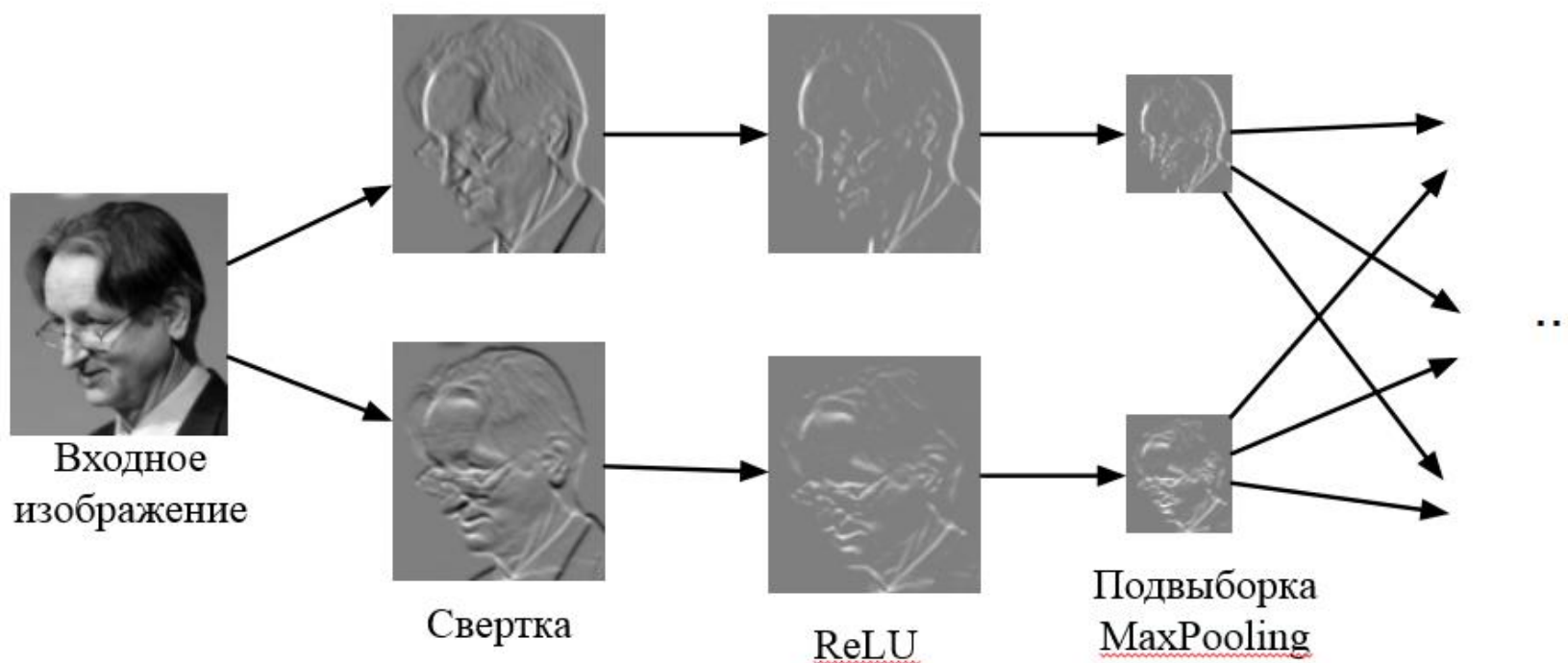


Функции активации

- Функция активации (англ. activation function) $a(x)$ определяет выходное значение нейрона в зависимости от результата взвешенной суммы входов и порогового значения.
- Rectified Linear Unit (ReLU)— это наиболее часто используемая функция активации при глубоком обучении. Данная функция возвращает 0, если принимает отрицательный аргумент, в случае же положительного аргумента, функция возвращает само число. То есть она может быть записана как $f(z)=\max(0,z)$
- В сетях с очень большим количеством нейронов использование ReLU, позволяет уменьшить количество включаемых нейронов в силу характеристик функции, и сама сеть станет легче.
- Недостаток: так как часть производной функции равна нулю, то и градиент для нее будет нулевым, а то это значит, что веса не будут изменяться во время спуска и нейронная сеть перестанет обучаться.

Примеры слоев сверточной сети

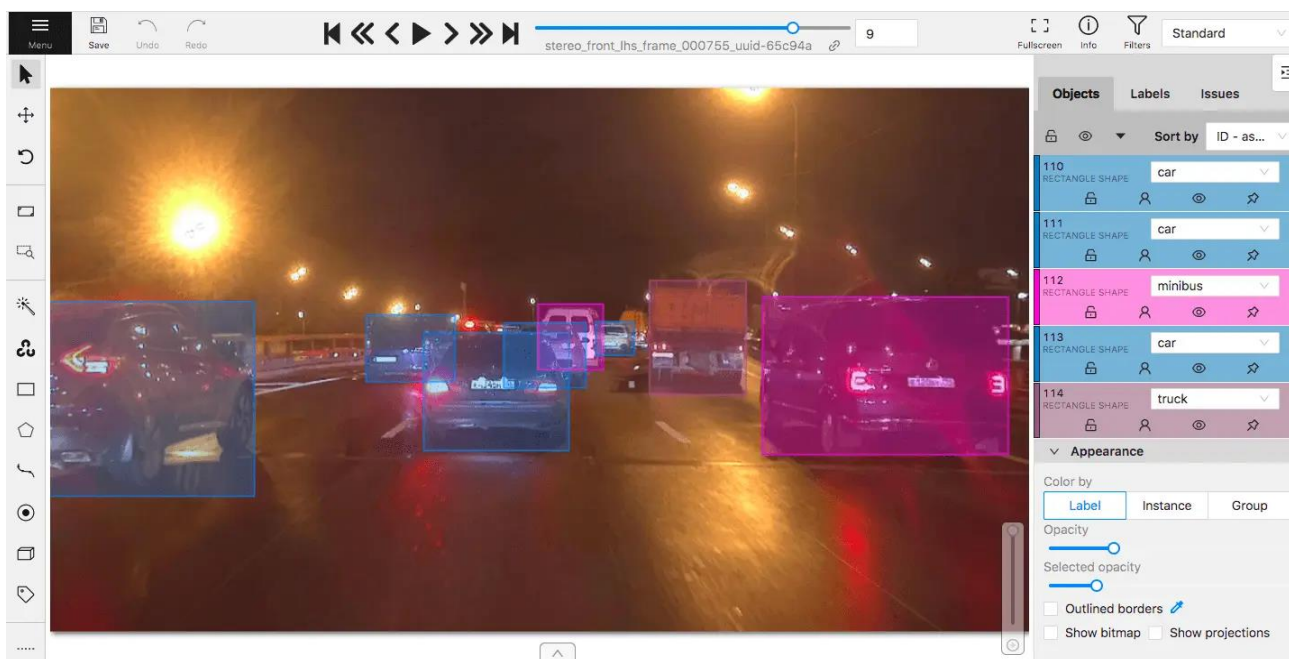
Операция свертки + ReLU + подвыборка



Подготовка данных

Прямоугольная разметка.

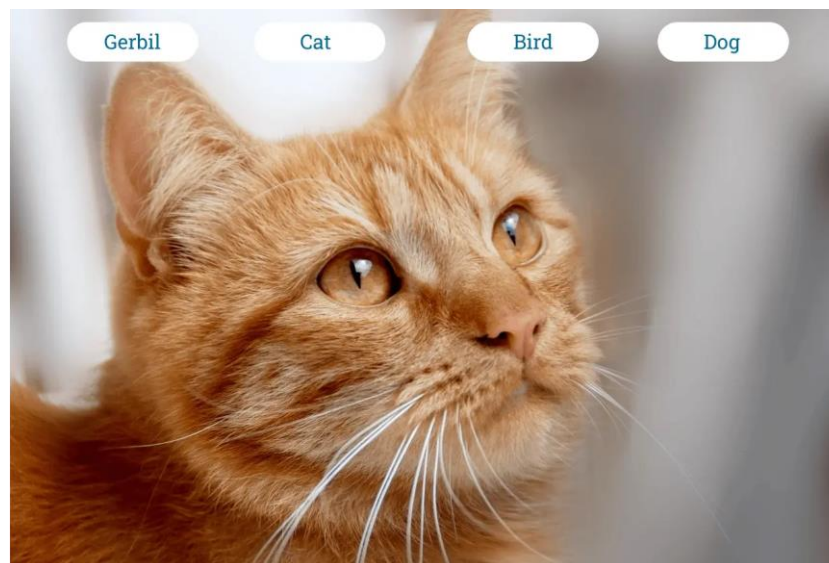
- Самый быстрый метод позволяет значительно сократить время, затраченное на обработку.
- В формате сложных проектов, где требуется высокая точность выходных данных, подобный вид разметки может оказаться недостаточным и снизит качество работы итоговых алгоритмов.



Подготовка данных

Проставление меток

- Позволяет создать алгоритм, способный выделять несколько свойств объекта согласно проставленным на нем мульти-меткам.
- Один объект может обладать несколькими различными свойствами, определяемыми программой, что соответственно будет причислять его как подходящий к нескольким группам.
- Дает возможность создать более быстрый процесс подготовки данных для работы алгоритма, в котором в зависимости от условия объект будет или не будет учтен, но имеет одну из самых больших погрешностей в качестве работы итоговой системы



Подготовка данных

Выделение полигонов

- На изображении полностью выделяются точные границы объектов, что позволяет нейронным сетям и алгоритмам получать наиболее «чистые» данные.
- При углубленной обработке изображений возможно также выделение рёбер, вершин и граней объекта, для создания 3D модели объекта.
- В ходе обработки изображения каждому пикселю присваивается своё значение, согласно которому алгоритм определяет границы объекта, а также его принадлежность к определенной группе.
- Два подхода
 - Все объекты, относящиеся к определенной группе, будут рассматриваться программой как один (semantic segmentation).
 - Метод случайной сегментации (instance segmentation), когда алгоритм распознает на изображении каждый отдельный объект вне зависимости от его отношения к конкретному классу.

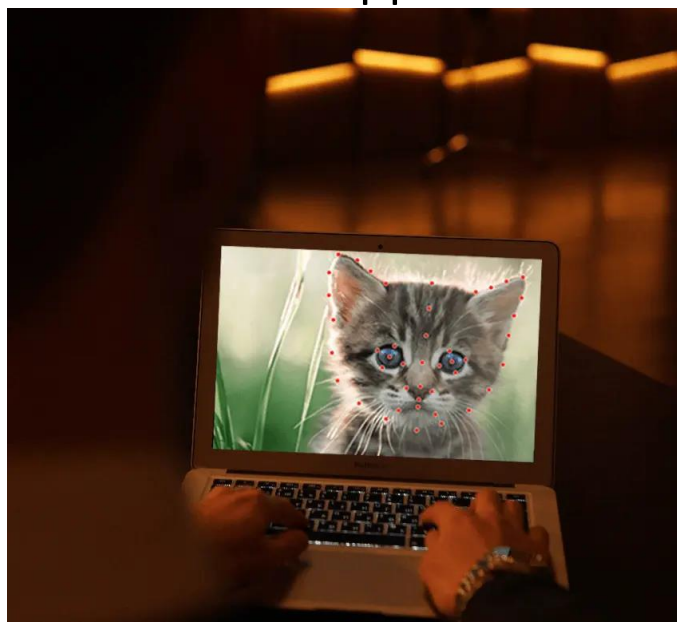
Подготовка данных



Подготовка данных

Множества точек.

- Данная разметка полезна для решения задач поиска ключевых точек тех или иных объектов, обычно применяется для решения задач классификации лиц: отличить одного человека от другого.
- Также применяется для определения позы человека, по координатам отдельных точек можно распознавать позу тела, положение пальцев и т.п. по видео



Этапы разметки данных

- 1. Подготовка исходных изображений или видео данных** - удаляется низкокачественный и дублированный контент перед отправкой на аннотирование.
- 2. Определение типов меток.** Для классификации изображений метки будут в виде номеров/названий классов, а для сегментации или обнаружения объектов — в виде масок и координат границ.
- 3. Создание класса для каждого аннотируемого объекта.** Количество классов должно быть фиксировано. Настройка меток и их названий на ранних этапах помогает предотвратить дублирование классов или метки похожих объектов под разными именами.
- 4. Аннотирование данных.** Область объекта можно аннотировать или добавлять метки изображения в зависимости от задачи. Сложные аннотации, такие как ограничивающие рамки, сегментные карты и полигоны, были максимально точными.
- 5. Версии и экспорт набора данных** Популярные методы экспорта - JSON, XML и pickle. Для обучения алгоритмов глубокого обучения используются форматы, такие как COCO и Pascal VOC.

Где брать данные

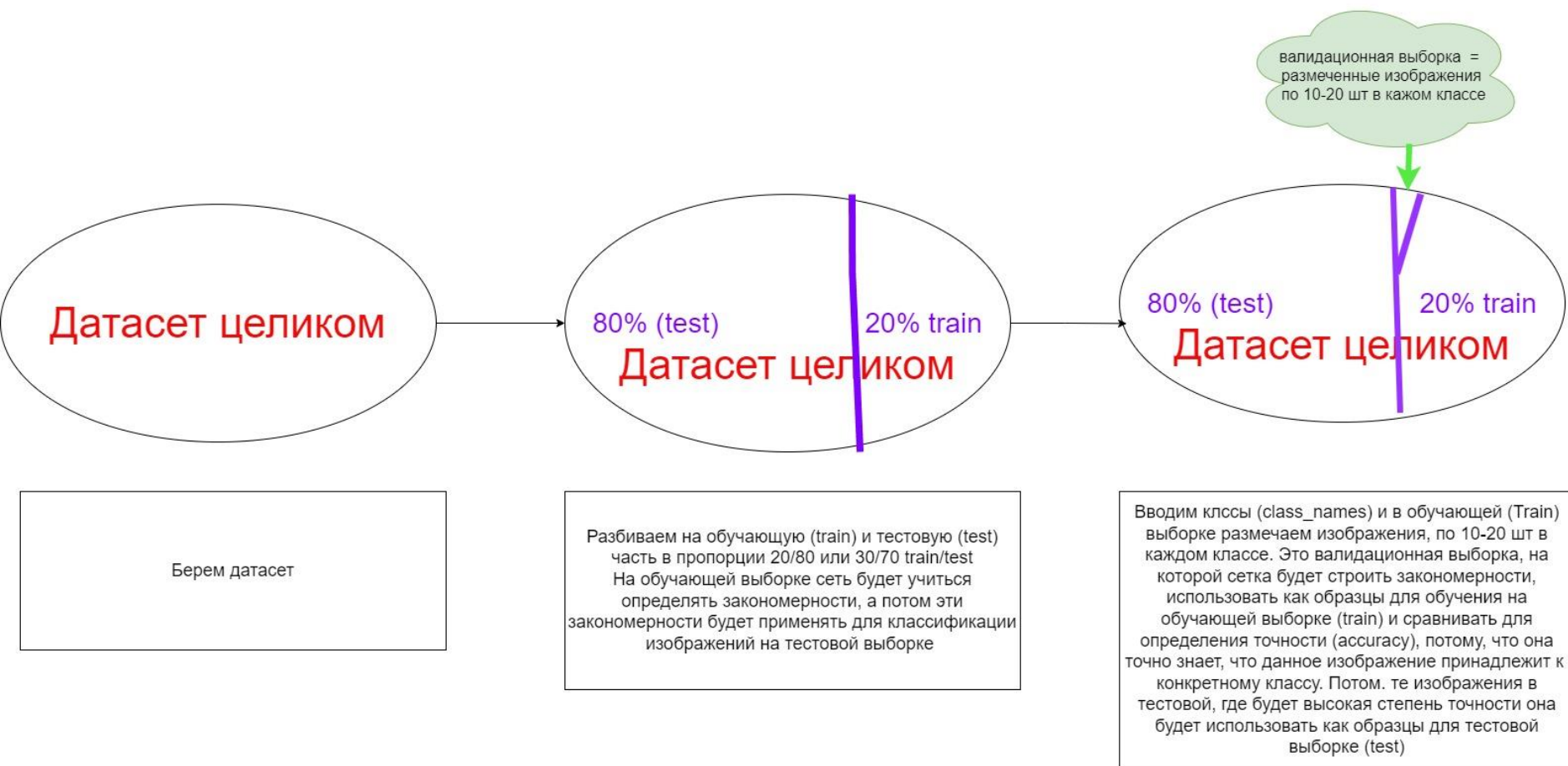
- <https://www.kaggle.com/>
- <http://vision.stanford.edu/>
- <https://image-net.org/about.php>
- <http://yann.lecun.com/exdb/mnist/>

Разметка и классификация изображений с помощью TensorFlow

- Загрузка и установка
<https://www.tensorflow.org/install?hl=ru>
- Можно использовать [Google Colaboratory](#) . Программы на Python запускаются непосредственно в браузере. Для начала - запускаем блокнот в Google Colab по ссылке
https://storage.googleapis.com/tensorflow_docs/docs/site/en/tutorials/quickstart/advanced.ipynb .
- Используем [Keras Sequential API](#)
(<https://www.tensorflow.org/guide/keras?hl=ru>)
- Импортируем TensorFlow

```
import tensorflow as tf from tensorflow.keras import datasets, layers, models import matplotlib.pyplot as plt
```

Разметка и классификация изображений с помощью TensorFlow



Разметка и классификация изображений с помощью TensorFlow

- Готовим и размечаем датасет

Пример разметка датасета [CIFAR10 dataset](#): 60,000 цветных изображений в 10 классах, по 6,000 изображений в каждом классе, разделен на 50,000 тренировочных изображений и 10,000 тестовых) Классы не пересекаются

```
(train_images, train_labels), (test_images, test_labels) =  
datasets.cifar10.load_data() # Normalize pixel values to be between 0 and 1  
train_images, test_images = train_images / 255.0, test_images / 255.0
```

Разметка и классификация изображений с помощью TensorFlow

- Верифицируем данные
- Берем первые 25 изображений тренировочного набора и размечаем название класса (категории) под каждым изображением

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',  
'frog', 'horse', 'ship', 'truck'] plt.figure(figsize=(10,10)) for i in  
range(25): plt.subplot(5,5,i+1) plt.xticks([]) plt.yticks([])  
plt.grid(False) plt.imshow(train_images[i]) # The CIFAR labels  
happen to be arrays, # which is why you need the extra index  
plt.xlabel(class_names[train_labels[i][0]]) plt.show()
```

Создаем сверточную сеть

- Создаем сверточный слой (пакет `tf.keras.layers.Conv2D`)
https://www.tensorflow.org/api_docs/python/tf/keras/layers/Conv2D

```
tf.keras.layers.Conv2D(  
    filters,  
    kernel_size,  
    strides=(1, 1),  
    padding='valid',  
    data_format=None,  
    dilation_rate=(1, 1),  
    groups=1,  
    activation=None,  
    use_bias=True,  
    kernel_initializer='glorot_uniform',  
    bias_initializer='zeros',  
    kernel_regularizer=None,  
    bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None,  
    bias_constraint=None,  
    **kwargs  
)
```

Создаем сверточную сеть

- Используем Max Pooling (пулинг-овый слой) пакет `tf.keras.layers.MaxPooling2D`

```
tf.keras.layers.MaxPooling2D(  
    pool_size=(2, 2),  
    strides=None,  
    padding='valid',  
    data_format=None,  
    **kwargs  
)
```

Создаем сверточную сеть

- На входе можно использовать тензоры высоты, ширины, цвета (`image_height`, `image_width`, `color_channels`), Раскладка по цвету (`color_channels`) - (R,G,B).
- В примере используются входы по размеру (32, 32, 3), - формат изображений CIFAR images. Используется `input_shape` для первого слоя

```
model = models.Sequential() model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3))) model.add(layers.MaxPooling2D((2, 2))) model.add(layers.Conv2D(64, (3, 3), activation='relu')) model.add(layers.MaxPooling2D((2, 2))) model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

- Отображение архитектуры

```
model.summary()
```

Создаем сверточную сеть

- Для завершения модели, добавляем последний исходящий тензор из сверточной базы (форма(4, 4, 64)) в полносвязный слой чтобы показать классификацию.
- Полносвязный слой превращает 3D тензоры в 1D. У CIFAR исходящих классов, так что нам нужен финальный полносвязный слой с 10 исходящими

```
model.add(layers.Flatten()) model.add(layers.Dense(64, activation='relu')) model.add(layers.Dense(10))
```

- Полная архитектура модели

```
model.summary()
```

Создаем сверточную сеть

- Компиляция и обучение

```
model.compile(optimizer='adam',  
loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),  
metrics=['accuracy']) history = model.fit(train_images, train_labels,  
epochs=10, validation_data=(test_images, test_labels))
```

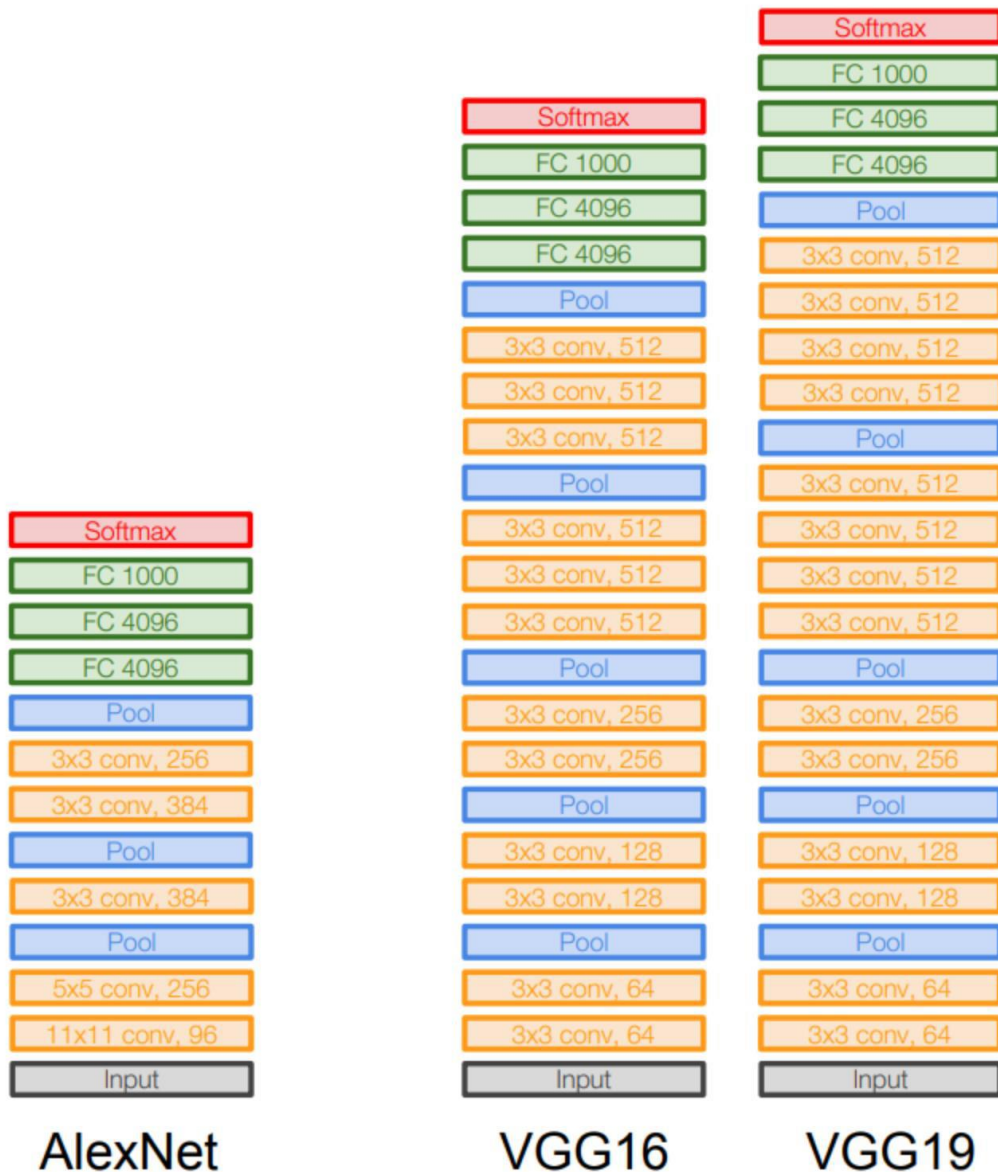
- Оценка модели

```
plt.plot(history.history['accuracy'], label='accuracy')  
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')  
plt.xlabel('Epoch') plt.ylabel('Accuracy') plt.ylim([0.5, 1])  
plt.legend(loc='lower right') test_loss, test_acc =  
model.evaluate(test_images, test_labels, verbose=2)  
print(test_acc)
```

Типы архитектуры сверточной сети

- Архитектура сверточной сети - варианты количества и комбинации слоев:
 - Backbone-часть (извлечение признаков)
 - Task-specific части (предсказание bounding boxов - выделение конкретного объекта или рисование рамки вокруг объекта на изображении, сегментирование объектов)

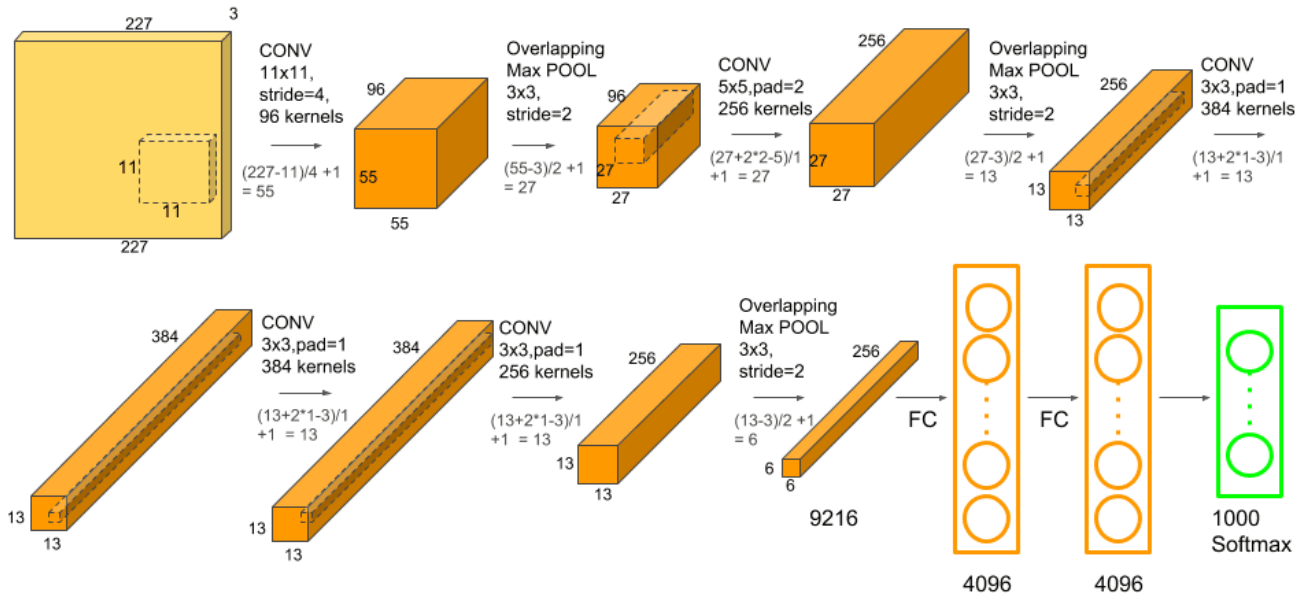
Виды Backbone архитектуры



- AlexNet
- VGG (16 и 19)
- ResNet
- Residual Block
- ResNeXt

AlexNet

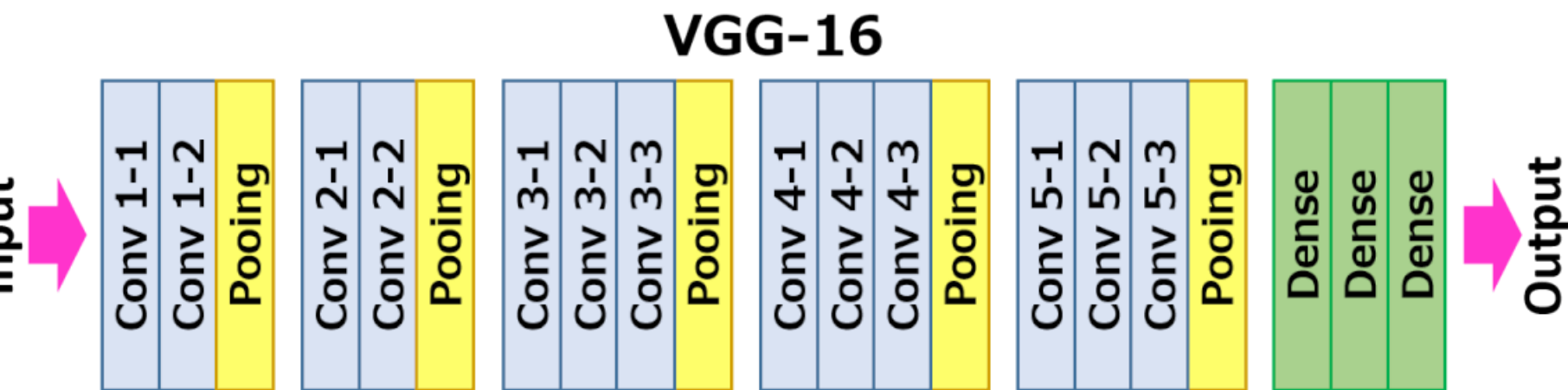
- 8 слоев, 90 эпох, как функция активации используется Relu
- Использование дропаута вместо регуляризации решает проблему переобучения (удваивает время обучения)
- Производится перекрытие объединений для уменьшения размера сети. (уровень ошибок первого и пятого уровней снижаются до 0,4% и 0,3%, соответственно).



[Ссылка](#) в Colab
[Ссылка](#) в ГитХаб
[Ссылка](#) для
Tensorflow

VGG16

- VGG16 — улучшенная версия AlexNet, в которой заменены большие фильтры (размера 11 и 5 в первом и втором сверточном слое, соответственно) на несколько фильтров размера 3x3, следующих один за другим.
- [Pytorch](#)
- [Tensorflow](#)
- [Keras](#)



VGG16

- На вход слоя conv1 подаются RGB изображения размера 224x224.
- Изображения проходят через стек сверточных слоев, в которых используются фильтры с очень маленьким рецептивным полем размера 3x3
- В одной из конфигураций используется сверточный фильтр размера 1x1, который может быть представлен как линейная трансформация входных каналов (с последующей нелинейностью).
- Сверточный шаг фиксируется на значении 1 пиксель. Пространственное дополнение (padding) входа сверточного слоя выбирается таким образом, чтобы пространственное разрешение сохранялось после свертки, то есть дополнение равно 1 для 3x3 сверточных слоев.
- Пространственный пулинг осуществляется при помощи пяти max-pooling слоев, которые следуют за одним из сверточных слоев (не все сверточные слои имеют последующие max-pooling). Операция max-pooling выполняется на окне размера 2x2 пикселей с шагом 2.
- После стека сверточных слоев идут три полносвязных слоя: первые два имеют по 4096 каналов, третий — 1000 каналов. Последним идет soft-max слой. Конфигурация полносвязных слоев одна и та же во всех нейросетях.
- Все скрытые слои снабжены ReLU.

VGG19

VGG-16	VGG-19
input (224x244x3)	
conv3-64 conv3-64	conv3-64 conv3-64
maxpool	
conv3-128 conv3-128	conv3-128 conv3-128
maxpool	
conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool	
conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool	
conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool	
FC-4096	
FC-4096	
FC-1000	
softmax	

- [Ссылка](#) на Tensorflow
- [Ссылка](#) на Keras
- [Ссылка](#) Гит
- Предобученная модель на [Гит](#)
- [Статья](#)

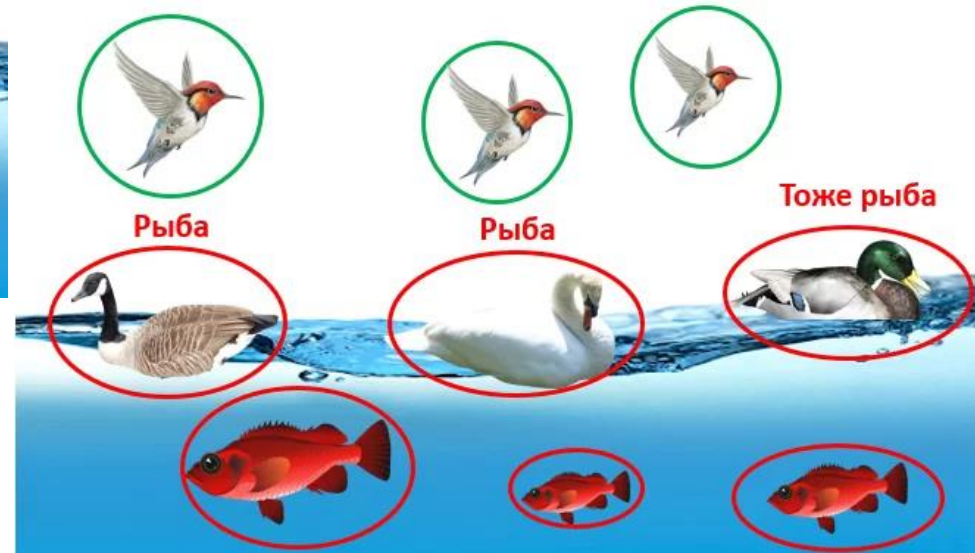
Transfer Learning

- Transfer Learning (трансферное обучение) — это подраздел машинного обучения, целью которого является применение знаний, полученные из одной задачи, к другой целевой задаче.
- Условия
 - Наличие открытой предварительно обученной модели
 - Переназначение модели с помощью выделения признаков (Feature extraction) или дообучения (Fine Tuning)

Переобучение и недообучение сетей



Все, что плавает = рыба, все
что летает = птица



Проблема – правило (признак), которое *имеет место только на обучающей выборке*. Такая ситуация и называется проблемой переобучения (overfitting).

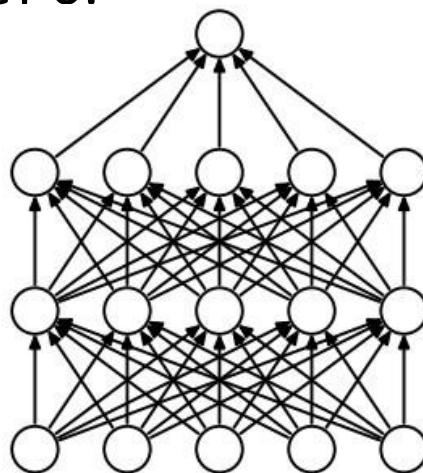
Переобучение – это явление, когда качество модели на обучающей выборке существенно превосходит качество модели на тестовой выборке.

Переобучение и недообучение сетей

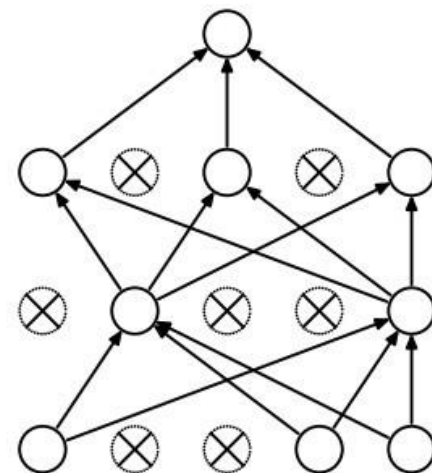
- Преобучение (overfitting) — проблема нейронных сетей – модель хорошо объясняет только примеры из обучающей выборки, адаптируясь к обучающим примерам, вместо того чтобы учиться классифицировать примеры, не участвовавшие в обучении (теряя способность к обобщению).
- **Переобучение (англ. overfitting)** — негативное явление, возникающее, когда алгоритм обучения вырабатывает предсказания, которые слишком близко или точно соответствуют конкретному набору данных и поэтому не подходят для применения алгоритма к дополнительным данным или будущим наблюдениям.
- **Недообучение (англ. underfitting)** — негативное явление, при котором алгоритм обучения не обеспечивает достаточно малой величины средней ошибки на обучающей выборке. Недообучение возникает при использовании недостаточно сложных моделей.

Dropout как метод борьбы с переобучением

- Dropout (в русскоязычных источниках — “метод прореживания”, “метод исключения” или просто “дропаут”).
 - Главная идея Dropout — вместо обучения одной сети обучить ансамбль нескольких DNN, а затем усреднить полученные результаты.
 - Сети для обучения получаются с помощью исключения из сети (dropping out) нейронов с вероятностью p таким образом, вероятность того, что нейрон останется в сети, составляет $q=1-p$. “Исключение” нейрона означает, что при любых входных данных или параметрах он возвращает 0.



(a) Standard Neural Net



(b) After applying dropout.

Dropout как метод борьбы с переобучением

- Исключенные нейроны не вносят свой вклад в процесс обучения ни на одном из этапов алгоритма обратного распространения ошибки (backpropagation), поэтому исключение хотя бы одного из нейронов равносильно обучению новой нейронной сети.
- В стандартной нейронной сети производная, полученная каждым параметром, сообщает ему, как он должен измениться, чтобы, учитывая деятельность остальных блоков, минимизировать функцию конечных потерь.
- Dropout предотвращает совместную адаптацию для каждого скрытого блока, делая присутствие других скрытых блоков ненадежным. Поэтому скрытый блок не может полагаться на другие блоки в исправлении собственных ошибок.
- Статья <https://habr.com/ru/companies/mvideo/articles/782360/>

Функции потерь

- Линейный классификатор состоит из двух частей: входных данных $\mathbf{X}$ и набора весов $\mathbf{W}$. Как именно нужно использовать обучающие данные, чтобы определить значения $\mathbf{W}$, с которыми классификатор будет работать лучше всего?
- Пусть обучающие данные состоят всего из трёх изображений и 3 исходных классов, а параметры $\mathbf{W}$ выбраны произвольно.
- Чтобы автоматически определить, какие значения $\mathbf{W}$ будут лучшими, нужен способ количественной оценки «непригодности» для любого $\mathbf{W}$. Функция потерь принимает на вход значения $\mathbf{W}$ и определяет, насколько плохо они работают



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

Функции потерь

- Есть набор из N значений обучающих данных $\mathbf{X}$ и $\mathbf{y}$, где $\mathbf{X}$ — исходные изображения (а точнее, значения их пикселей)
- $\mathbf{y}$ — те категории, которые алгоритм должен предсказать (метки классов или целевые переменные).
- В процессе классификации мы получаем оценки от 1 для каждой из категорий, которые показывают вероятность принадлежности изображения $\mathbf{x}_i$ к классу $\mathbf{y}_i$.

$$\{(x_i, y_i)\}_{i=1}^N$$

- Обозначим функцию потерь как L_i . Классификатор f берёт пример $\mathbf{x}_i$, весовую матрицу $\mathbf{W}$ и делает прогноз для класса $\mathbf{y}_i$.
- Функция потерь L_i обрабатывает полученные результаты, сравнивая их с заранее известными истинными метками. Это поможет оценить, насколько плохо или хорошо выполняется прогноз для текущего обучающего примера $\mathbf{x}_i$.
- Общие потери L можно посчитать как среднюю сумму потерь для всех обучающих данных:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Функции потерь

- Представим прогнозы классификатора (оценки) в виде векторов: $\mathbf{s} = \mathbf{f}(\mathbf{x}_i, \mathbf{W})$.
- Для каждого отдельного примера просуммируем значения оценок всех y , кроме истинной метки y_i . То есть, мы получим сумму значений всех неправильных категорий.
- Посчитаем разницу между ложными прогнозами и истинной оценкой: $s_j - s_{y_i}$; $j \neq y_i$.
- Если истинная оценка больше, чем сумма всех неправильных прогнозов плюс некоторый дополнительный «зазор» (установим его равным 1) — значит, полученный балл для правильной категории намного превосходит любую ошибочную оценку. Это и будет наша функция потерь.

$$\begin{aligned} L_i &= \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_j + 1 \\ s_j - s_{y_i} + 1 & \text{otherwise} \end{cases} \\ &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \end{aligned}$$

Функции потерь



$$s = f(x_i, W)$$

cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9		

$$\begin{aligned} L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \max(0, 5.1 - 3.2 + 1) \\ &\quad + \max(0, -1.7 - 3.2 + 1) \\ &= \max(0, 2.9) + \max(0, -3.9) \\ &= 2.9 + 0 \\ &= 2.9 \end{aligned}$$

Истинная метка для изображения — “cat”, её оценка — 3.2.

Мы вычитаем правильную оценку из всех неправильных и сравниваем: если разность больше нуля, значит, метка определена ошибочно и потери присутствуют, если меньше — всё хорошо, потери нулевые.

После этого суммируем результаты. Для класса “cat” значение функции потерь равно 2.9.

Функции потерь

- Среднеквадратическая ошибка - среднее значение квадратов разностей между прогнозируемыми и фактически наблюдаемыми значениями.

```
import numpy as np # импортируем библиотеку numpy
```

```
def mean_squared_error(act, pred): # функция
```

```
    diff = pred - act # находим разницу между прогнозируемыми и наблюдаемыми значениями
```

```
    differences_squared = diff ** 2 # возводим в квадрат (чтобы избавиться от отрицательных значений)
```

```
    mean_diff = differences_squared.mean() # находим среднее значение
```

```
    return mean_diff
```

```
act = np.array([1.1,2,1.7]) # создаем список актуальных значений
```

```
pred = np.array([1,1.7,1.5]) # список прогнозируемых значений
```

```
print(mean_squared_error(act,pred))
```

Функции потерь

- **Корень среднеквадратической ошибки (RMSE)** - используем для того чтобы избавиться от квадратной степени, в которую мы ранее возвели действительную ошибку среди между прогнозируемыми и фактически наблюдаемыми значениями.

```
import numpy as np
```

```
def root_mean_squared_error(act, pred):
```

```
    diff = pred - act # находим разницу между прогнозируемыми и наблюдаемыми значениями
```

```
    differences_squared = diff ** 2 # возводим в квадрат
```

```
    mean_diff = differences_squared.mean() # находим среднее значение
```

```
    rmse_val = np.sqrt(mean_diff) # извлекаем квадратный корень
```

```
    return rmse_val
```

```
act = np.array([1.1,2,1.7])
```

```
pred = np.array([1,1.7,1.5])
```

```
print(root_mean_squared_error(act,pred))
```

ResNet

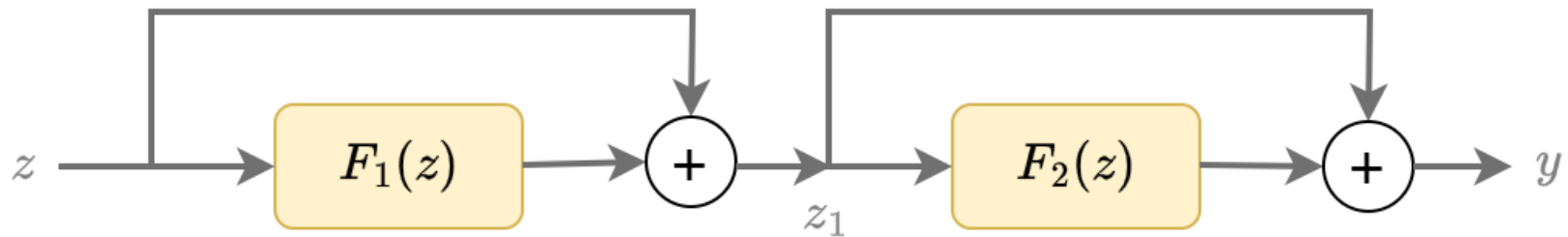
- ResNet (Residual Network) — это архитектура сверточных нейронных сетей, использует остаточные блоки (Residual Blocks), которые позволяют строить сверхглубокие сети (до 152 слоев) без проблемы затухающих градиентов.
- Проблема исчезающих градиентов возникает из-за того, что множество функций активации (таких, как сигмоида) сокращают большое входное пространство до гораздо меньшего выходного (например, от 0 до 1 для сигмoиды). Поскольку большие изменения входа преобразуются в намного меньшие изменения выхода, производная, естественно, становится намного меньшей.
- До разработки ResNets, наращивание количества слоев для повышения глубины нейронной сети рано или поздно вызывало ограничение или даже быстрое ухудшение производительности сети.
- Стала основой для многих других архитектур, таких как ResNeXt, DenseNet и EfficientNet, которые также используют остаточные блоки и прямые переходы для достижения лучшей производительности
- Статья <https://arxiv.org/pdf/1512.03385>

ResNet. Остаточный блок

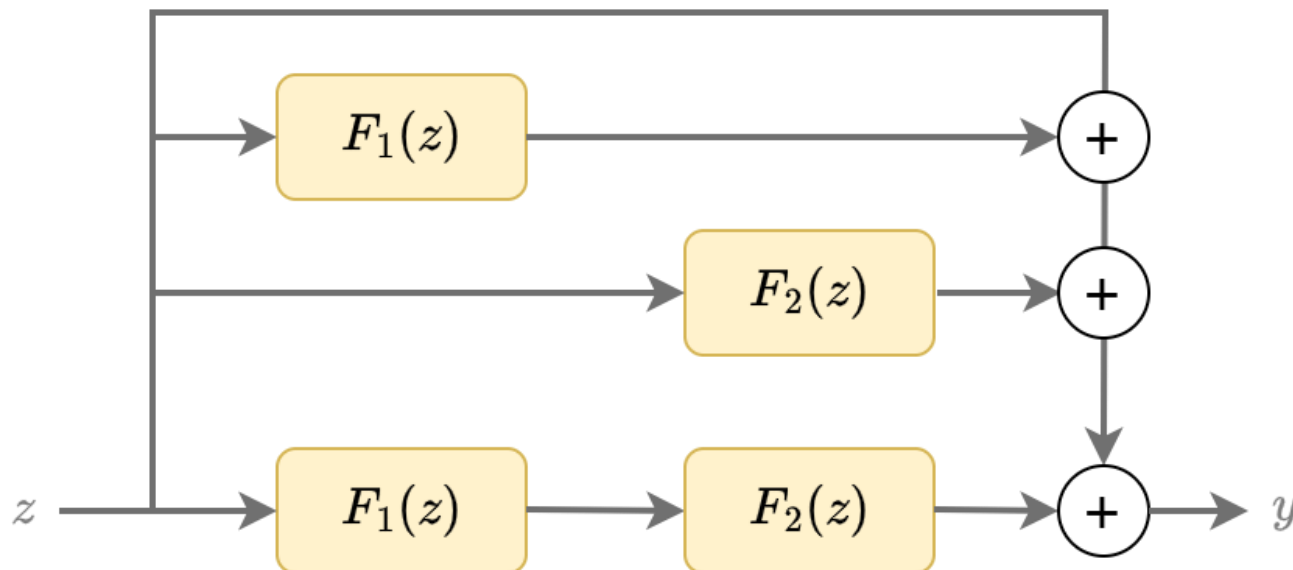
- **Резидуальные связи (skip connections)**, которые позволяют пропускать информацию через один или несколько слоев. Эти связи позволяют передавать прямую информацию от одного слоя к следующему, минуя промежуточные слои, что помогает избежать затухания градиента и улучшает обучение очень глубоких сетей. Выход одного или нескольких слоев добавляется к выходу следующего слоя, что позволяет сети лучше сохранять и передавать важную информацию, даже если сеть очень глубокая.
- Основным строительным блоком ResNet является **резидуальный блок**. Этот блок включает в себя несколько сверточных слоев с функцией активации ReLU, к которым добавляется прямое соединение, проходящее через весь блок. Такая конструкция помогает глубокой сети сохранять производительность и улучшать извлечение признаков. Обычно в блоке используются сверточные слои с фильтрами 3×3 , хотя могут быть и другие конфигурации.
- **Глубокие сети.** ResNet позволила строить сети с очень большим числом слоев, например, ResNet-50, ResNet-101 и ResNet-152 . Эти сети значительно глубже, чем предыдущие архитектуры, такие как VGG, и при этом обучаются эффективно благодаря резидуальным связям.
- **Использование сверточных слоев 1×1 .** В некоторых вариантах резидуальных блоков используются сверточные слои с фильтрами 1×1 , чтобы изменить размерность каналов признаков и оптимизировать вычисления.

ResNet. Остаточный блок

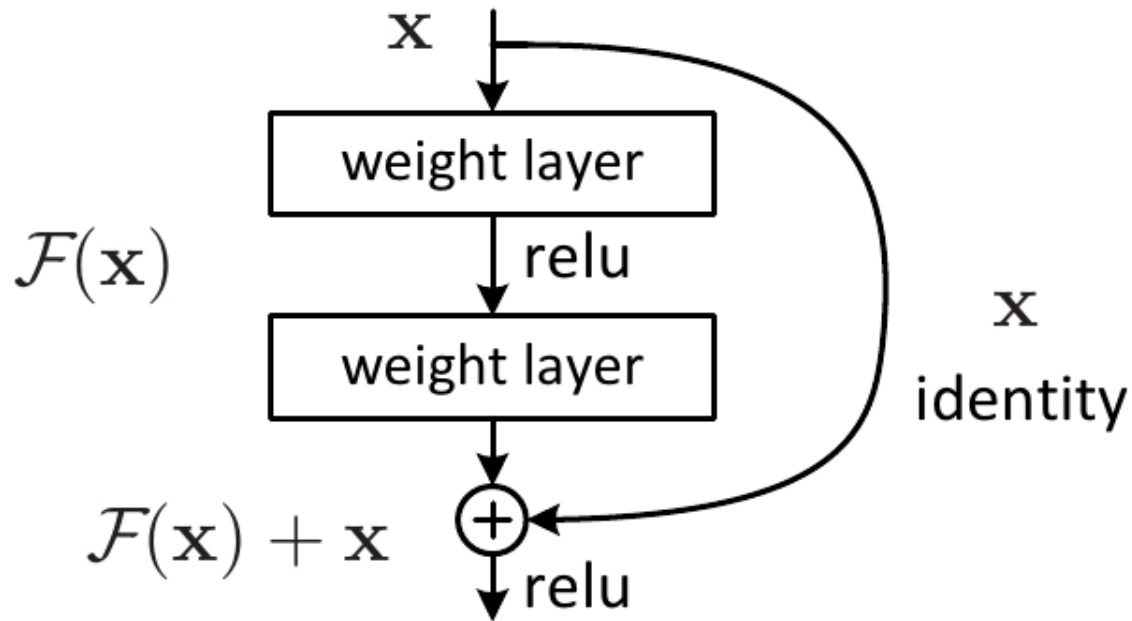
- На практике используется не один, а сразу несколько подряд идущих остаточных блоков, как показано ниже:



что графически представляется как одновременное воздействие на выход менее и более глубоких сетей, т.е. ансамбль моделей:



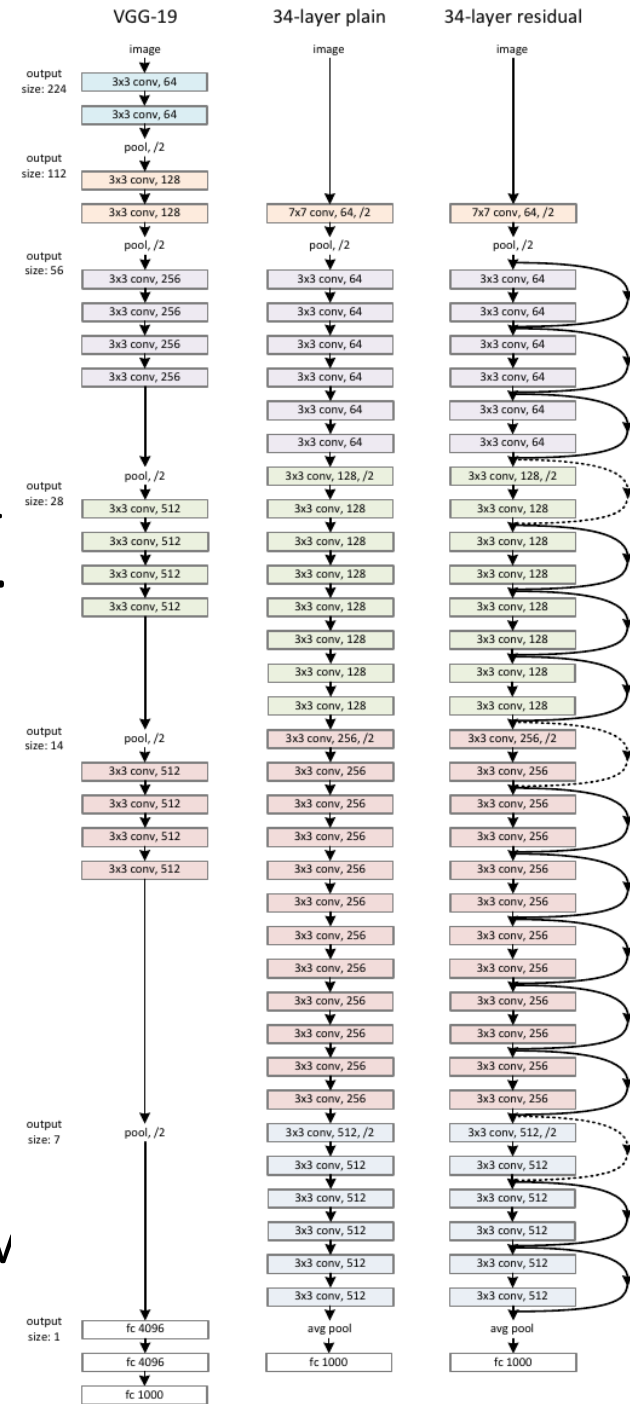
ResNet



- Результатом действия остаточного блока является сумма нелинейного преобразования и исходного входа $\mathcal{F}(\mathbf{x}) + \mathbf{x}$.
- Такой конструкции гораздо проще воспроизвести тождественное преобразование, для чего достаточно задать нулями все веса преобразования $\mathcal{F}(\mathbf{x})$.
- После каждого остаточного блока в ResNet действует нелинейность [ReLU](#).

ResNet

- Для суммирования преобразования $F(x)$ и входа x необходимо, чтобы число каналов и пространственные размеры в результате преобразования не изменялись. Поэтому в $F(x)$ использовались свёртки 3×3 с шагом 1 и расширением (padding), равным единице.
- Периодически пространственное разрешение уменьшалось в два раза, что реализовывалось свёрткой с шагом 2 в преобразовании $F(x)$, а в тождественной связи применялась свёртка 1×1 также с шагом 2.
- Сравнение архитектуры VGG-19 (слева), её расширенная версия с большим числом слоёв (в центре) и она же, но с добавлением тождественных связей (справа)



ResNet

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Нелинейные преобразования $F(x)$ различались для числа слоёв 34, 50 и 101, 152. В первом случае они реализовывались двумя свёртками с пространственными размерами (размером ядра) 3×3, а во втором - тремя свёртками 1×1, 3×3, 1×1. Сначала 1×1 для сокращения размерности, затем 3×3 для свертки, и снова 1×1 для увеличения размерности обратно. Снижает количество вычислений и параметров при сохранении высокой производительности

ResNet. Преимущества

- **Тожественные связи** позволяют градиенту лучше распространяться **назад** при обучении модели, ускоряя настройку более ранних слоёв нейросети.
- **Тожественные связи существенно упрощают начальную инициализацию глубокой сети:** веса нелинейных преобразований нужно инициализировать значениями, близкими к нулю, чтобы в начале оптимизации $F(x) \approx 0$. Если оптимизатор сети сочтёт, что сети не хватает выразительной способности для обработки сложных объектов, он настроит $F(x)$ нужным образом, причём только для тех блоков, которые реально необходимы.
- **ResNet позволяет одновременно обрабатывать как сложные объекты, так и простые объекты.** Для распознавания первых нужно много слоёв, а для вторых - мало. В последнем случае признаки извлекаются из сигнала, который прошёл через небольшое число нелинейных преобразований, а дальше распространялся в основном по тождественным связям.
- **ResNet действует как ансамбль**, поскольку каждый входной сигнал обрабатывается не по одной ветке обработки, а сразу по набору альтернативных веток, причем число альтернатив растёт экспоненциально с увеличением числа остаточных блоков.

ResNet. Код

- [В Tensorflow](#) (так себе, просто подгрузка библиотек)
- Git ([хорошая, с объяснениями](#))

Лабораторная работа 2

Рекуррентные нейронные сети

Что делать по лабораторной? (Задачи 1-2)

- Формируем датасет – временную последовательность. Берем из kaggle, [например](#) или [еще одну](#)

, подготавливаем (нормализуем) –

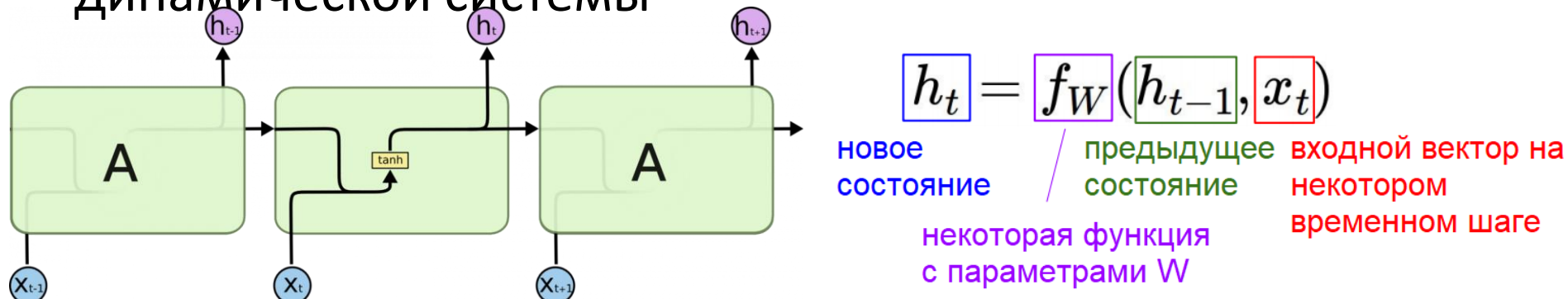
[хорошая статья по обработке временных рядов](#)

- На их основе строим три архитектуры RNN (простую, LSTM, GRU)
- Используем [встроенные возможности Keras](#) (RNN + RNN API)
<https://ru-keras.com/recurrent-layers/>

- В TensorFlow 2.0 есть базовый функционал для построения
https://www.tensorflow.org/api_docs/python/tf/keras/layers/RNN ,
встроенные слои LSTM и GRU пригодны для использования ядер CuDNN по умолчанию, если доступен графический процессор.
<https://habr.com/ru/articles/487808/>

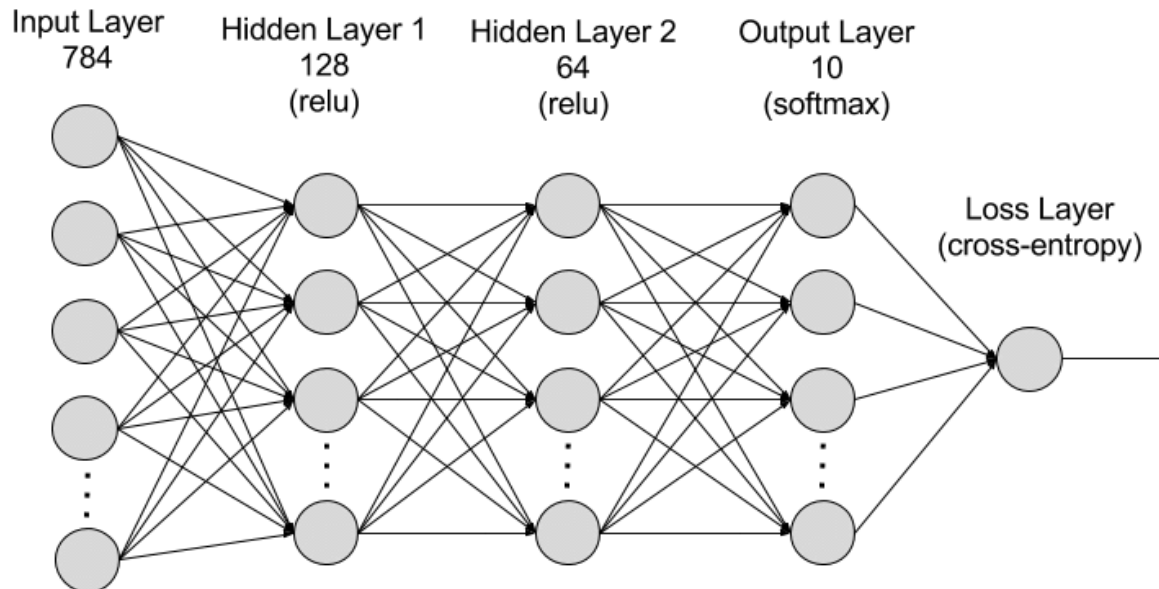
Рекуррентные нейронные сети

- Рекуррентные нейронные сети (Recurrent Neural Networks, RNN) – сети с обратными или перекрестными связями между различными слоями нейронов
- Изначально предложены для обработки:
 - последовательностей однотипных данных, предоставления объектов сети имеет важное значение
- Типовые примеры задач:
 - распознавание речи: обработка последовательности звуков, обработка текстов естественного языка
 - компьютерное зрение: обработка последовательности кадров видео, некоторые задачи обработки изображений
 - прогнозирование продаж, спроса, отклика от рекламы
- Рекуррентная сеть аппроксимирует поведение любой динамической системы



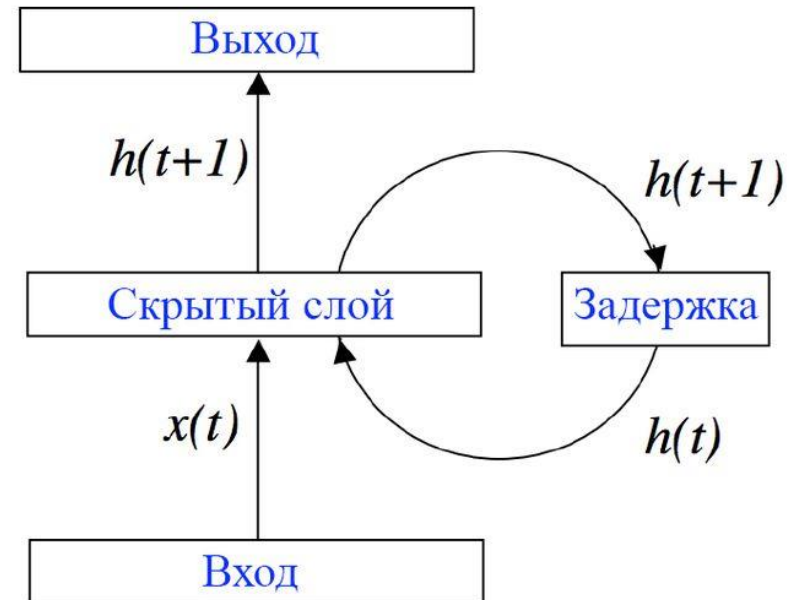
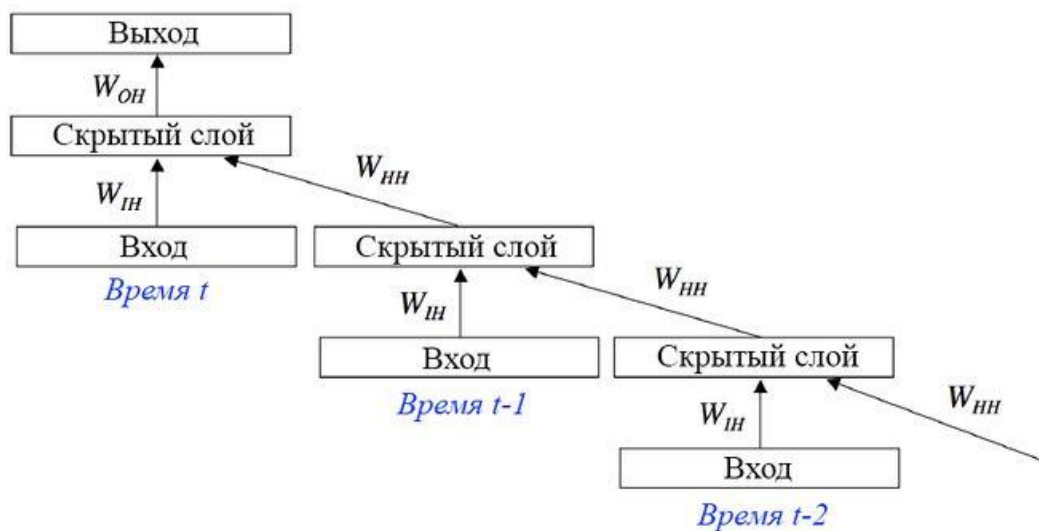
Как работает?

- Нейроны распределяются по входному, выходному и нескольким скрытым слоям
- Входной слой получает информацию для обработки, а выходной слой возвращает результат
- Обработка, анализ и прогнозирование данных происходят в **скрытых слоях**
- Использование алгоритма обратного распространения ошибки «сквозь время» (Backpropagation Through Time, BPTT)



Скрытые слои

- Скрытый слой может запоминать предыдущие входные данные в компоненте краткосрочной памяти и использовать их для будущих прогнозов.
- Модель использует текущее входное значение и сохраненное в памяти значение для прогнозирования следующей последовательности.
- Пример - последовательность *Apple is red*. Надо, чтобы RNN предсказывала слово *red* при получении входной последовательности *Apple is*. Когда скрытый слой обрабатывает слово *Apple*, он сохраняет его копию в своей памяти. Затем, увидев слово *is*, модель извлекает слово *Apple* из памяти и понимает контекст всей последовательности: *Apple is*. Это позволяет высокой точностью прогнозировать слово *red*.



Алгоритм обратного распространения ошибки

- Рассмотрим простую нейронную сеть без скрытых слоев, с двумя входными вершинами и одной выходной, в которых каждый нейрон использует линейную функцию активации взвешенную сумму входных данных.
- Простая нейронная сеть с двумя входными вершинами и одной выходной
- Изначально веса задаются случайно. Затем, нейрон обучается с помощью тренировочного множества, которое в этом случае состоит из множества троек (x_1, x_2, t) , где x_1 и x_2 — это входные данные сети и t — правильный ответ. Начальная сеть, приняв на вход x_1 и x_2 , вычислит ответ y , который вероятно отличается от t .
- Общепринятый метод вычисления несоответствия между ожидаемым t и получившимся y ответом — квадратичная функция потерь:

$$E = (t - y)^2, \text{ где } E \text{ ошибка}$$

Алгоритм обратного распространения ошибки

η - скорость обучения

α - коэффициент инерциальности для сглаживания резких скачков при перемещении по поверхности целевой функции

$\{x_i^d, t^d\}_{i=1, d=1}^{n, m}$ обучающее множество

steps - количество повторений

$network(x)$ функция, подающая x на вход сети и возвращающая выходные значения всех ее узлов

layers - количество слоев в сети

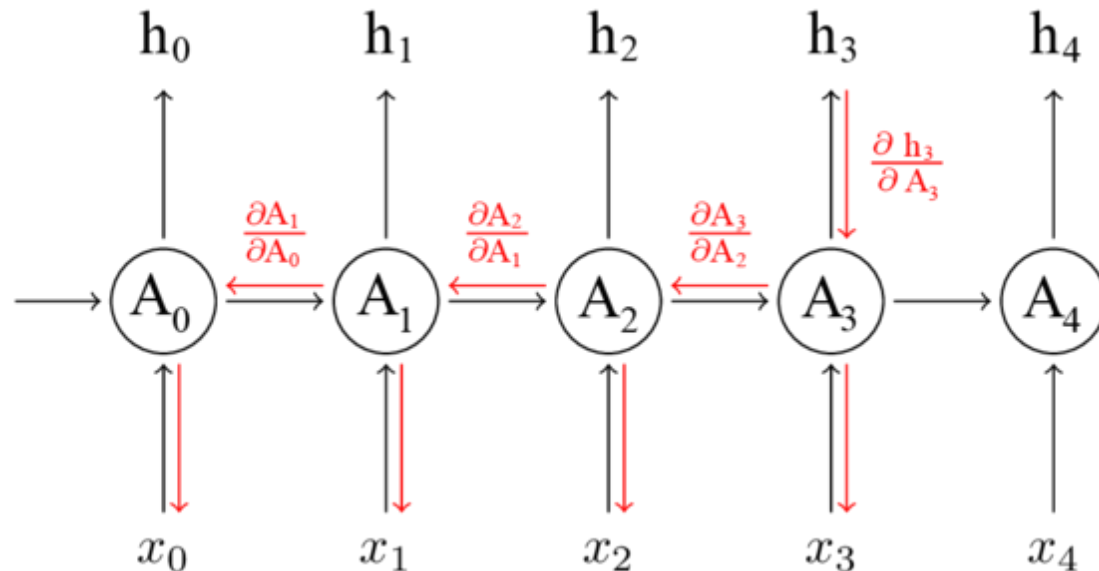
layer _{i} - множество нейронов в слое i

output - множество нейронов в выходном слое

```
fun BackPropagation( $\eta, \alpha, \{x_i^d, t^d\}_{i=1, d=1}^{n, m}, steps$ ):  
    init  $\{w_{ij}\}_{i, j}$   
    repeat steps:  
        for  $d = 1$  to  $m$ :  
             $o = network(\{x_i^d\})$   
            for  $k \in output$ :  
                 $\delta_k = \sigma'(o_k)(t_k - o_k)$   
            for  $i = layers - 1$  to  $1$ :  
                for  $j \in layer_i$ :  
                     $\delta_j = \sigma'(o_j) \sum_{k \in layer_{i+1}} \delta_k w_{j, k}$   
            for  $\forall w_{i, j}$ :  
                 $\Delta w_{i, j}^n = \alpha \Delta w_{i, j}^{n-1} + (1 - \alpha) \eta \delta_j o_i$   
                 $w_{i, j}^n = w_{i, j}^{n-1} + \Delta w_{i, j}^n$   
    return  $w_{ij}$ 
```

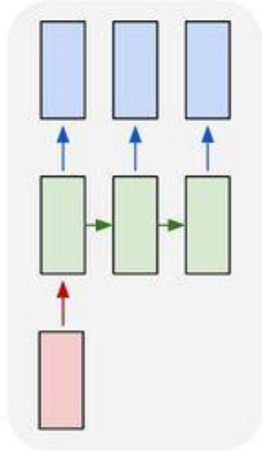
Алгоритм обратного распространения ошибки сквозь время» (Backpropagation Through Time, BPTT)

- Поскольку одни и те же параметры используются на всех временных этапах в сети, градиент на каждом выходе зависит не только от расчетов текущего шага, но и от предыдущих временных шагов (рис. 4).
- Например, чтобы вычислить градиент для четвертого элемента последовательности, нам нужно было бы «распространить ошибку» на 3 шага и суммировать градиенты.



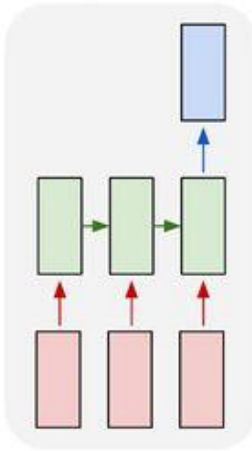
Виды RNN

Один ко многим



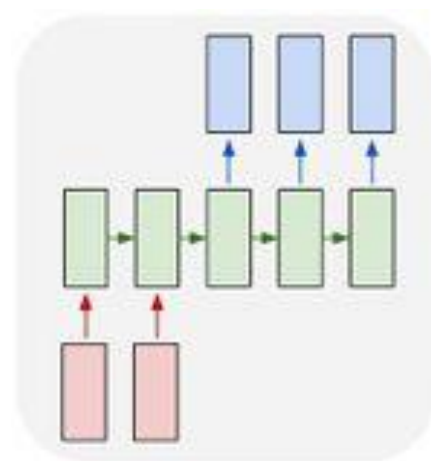
- Один вход ко многим выходам может применяться для генерации аудиозаписи.
- На вход подаем жанр музыки, который хотим получить, на выходе получаем последовательность аудиозаписи

Многие к одному



- Много входов и один выход может применяться, если мы хотим оценить тональность рецензии. На вход подаем слова рецензии, на выходе получаем оценку: позитивная рецензия или негативная

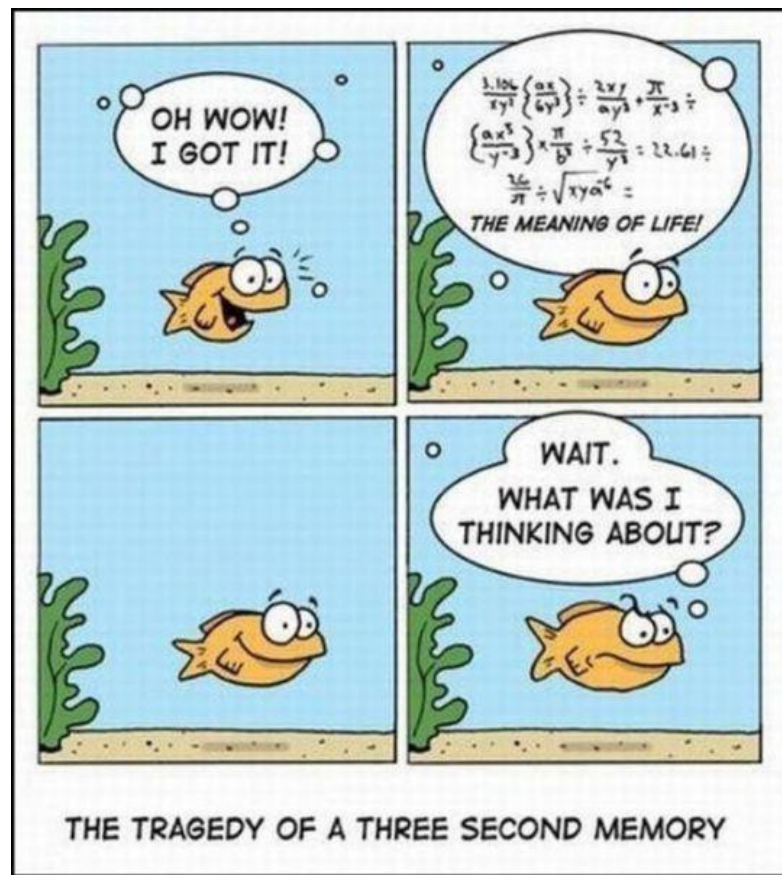
Многие ко многим



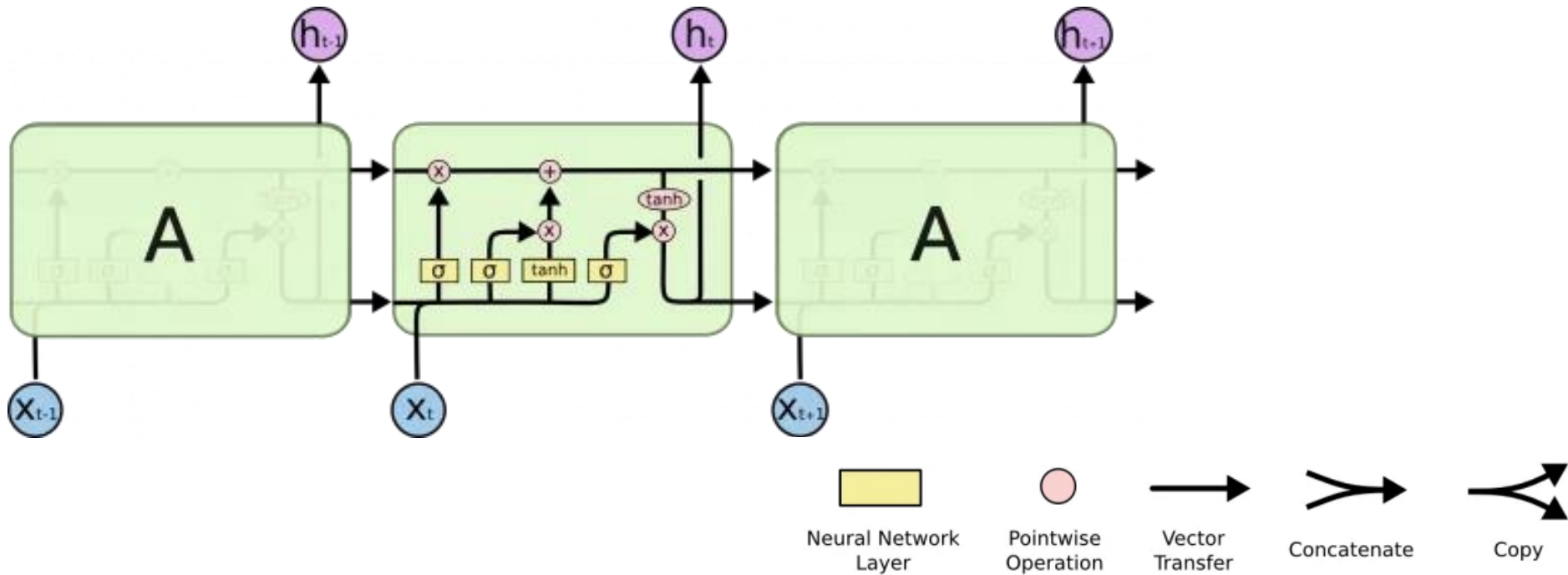
- Можно использовать для перевода текста с одного языка на другой.

Виды архитектуры RNN. LSTM

- Обычная RNN запоминает только самое последнее входное значение. Он не умеет использовать входные данные из нескольких предыдущих последовательности, чтобы улучшить точность прогнозирования.
- Долговременная кратковременная память (LSTM) — это вариация RNN, в которой объем памяти для модели увеличивается для размещения данных за более длительный период.



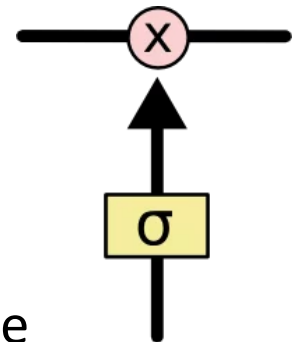
Виды архитектуры RNN. LSTM



- Каждая линия является вектором.
- Розовый круг - поточечные операции, например, суммирование векторов.
- Желтые ячейки - слои нейронной сети.
- Совмещение линий - объединение векторов,
- Знак разветвления — копирование вектора с последующим хранением в разных местах

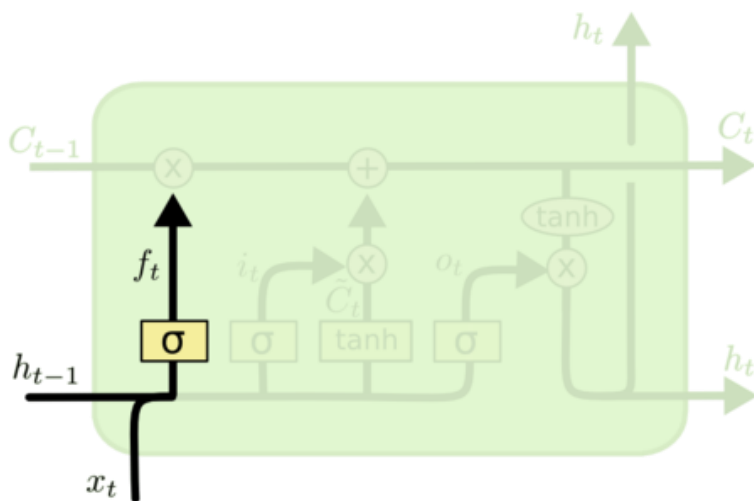
Виды архитектуры RNN. LSTM

- Ключевым понятием LSTM является состояние ячейки: горизонтальная линия, проходящая через верхнюю часть диаграммы.
- В LSTM уменьшает или увеличивает количество информации в состоянии ячейки, в зависимости от потребностей. Для этого используются тщательно настраиваемые структуры, называемые гейтами.
- Гейт — это «ворота», пропускающие или не пропускающие информацию. Гейты состоят из сигмовидного слоя нейронной сети и операции поточечного умножения (когда над двумя функциями f и g с одинаковой областью определения образуют новую функцию, значения которой являются результатом применения указанной бинарной операции к значениям f и g)
- На выходе сигмовидного слоя выдаются числа от нуля до единицы, определяя, сколько процентов каждой единицы информации пропустить дальше. Значение «0» означает «не пропустить ничего», значение «1» — «пропустить все».



Виды архитектуры RNN. LSTM

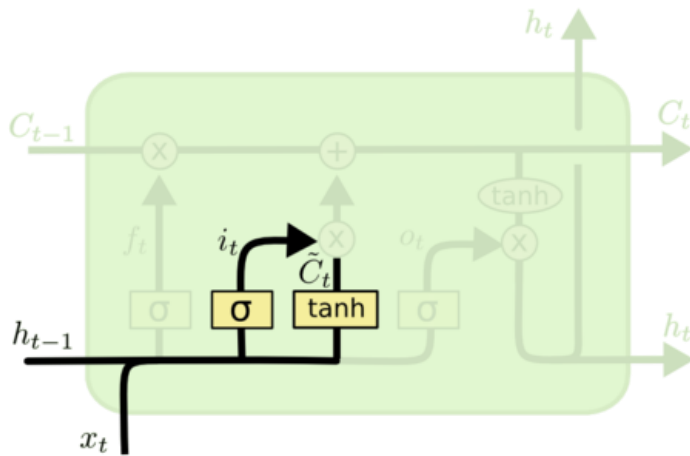
- Слой утраты “слой фильтра забывания” (англ. forget gate layer) определяет, какую информацию можно забыть или оставить. Значения предыдущего выхода h_{t-1} и текущего входа x_t пропускаются через сигмоидальный слой. Полученные значения находятся в диапазоне $[0; 1]$. Значения, которые ближе к 0 будут забыты, а к 1 оставлены.



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Виды архитектуры RNN. LSTM

- Слой сохранения (слой входного фильтра” (англ. input layer gate) определяет, какие значения следует обновить. Затем tanh-слой строит вектор новых значений-кандидатов $\tilde{C}_t$, которые можно добавить в состояние ячейки.

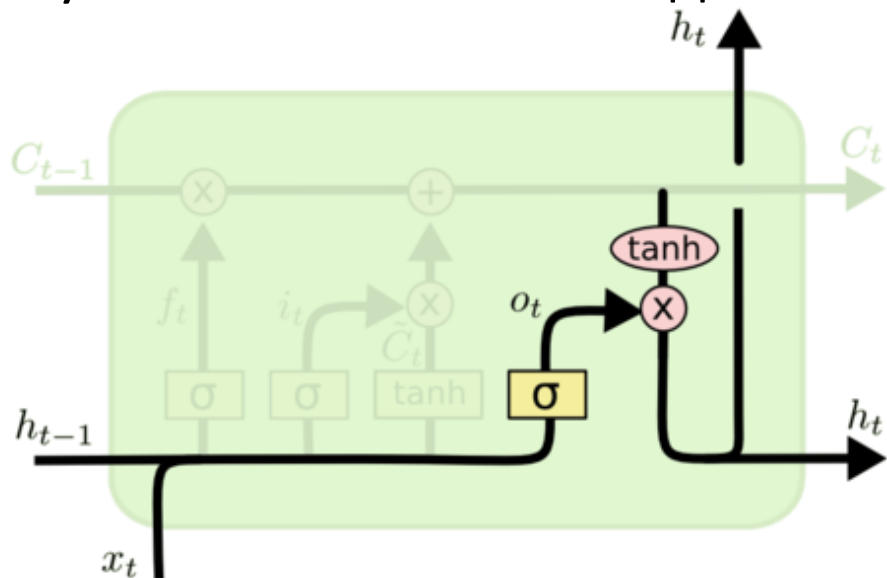


$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Виды архитектуры RNN. LSTM

- Выходной слой (слой нового состояния) нужно решить, что хотим получить на выходе. Результат будет являться отфильтрованным состоянием ячейки. Сначала запускаем сигмоидный слой, который решает, какие части состояния ячейки выводить. Затем пропускаем состояние ячейки через гиперболическую функцию $\tanh$ (чтобы разместить все значения в интервале $[-1, 1]$) и умножаем его на выходной сигнал сигмовидного гейта.

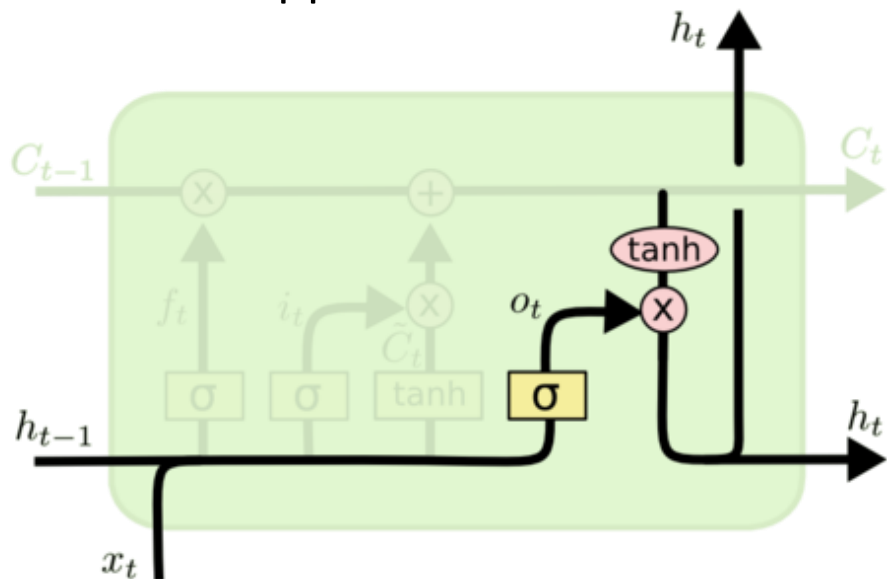


$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Виды архитектуры RNN. LSTM

- Выходной слой (слой нового состояния) нужно решить, что хотим получить на выходе. Результат будет являться отфильтрованным состоянием ячейки. Сначала запускаем сигмоидный слой, который решает, какие части состояния ячейки выводить. Затем пропускаем состояние ячейки через $\tanh$ (чтобы разместить все значения в интервале $[-1, 1]$) и умножаем его на выходной сигнал сигмовидного гейта.

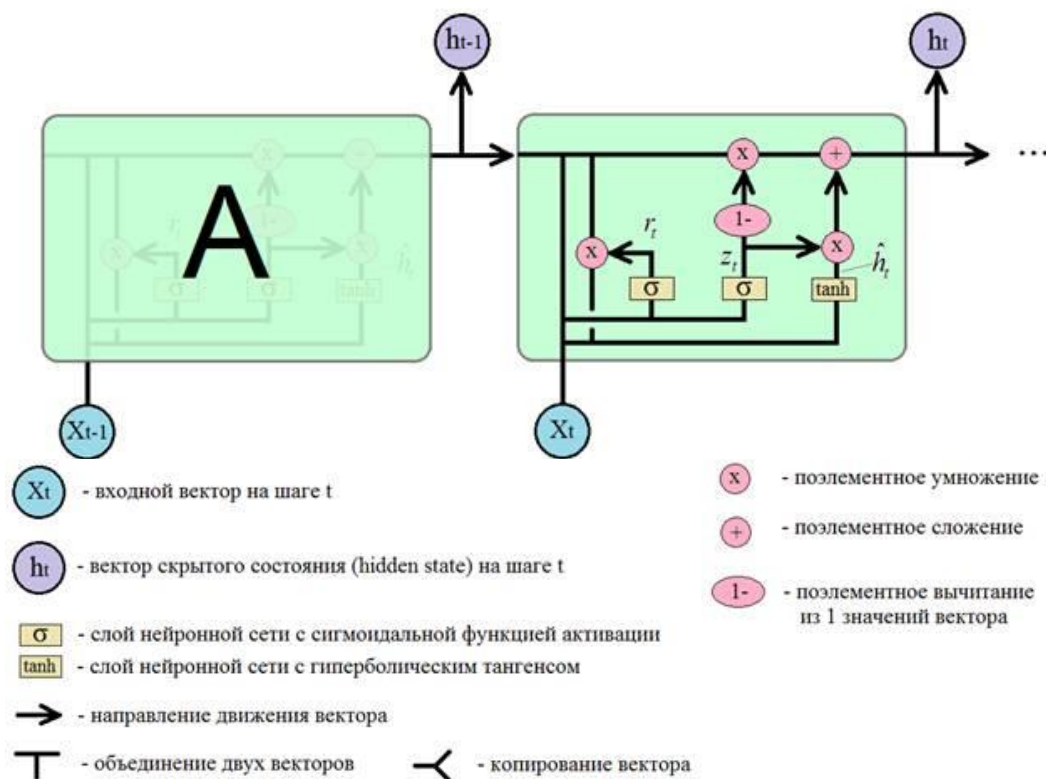


$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

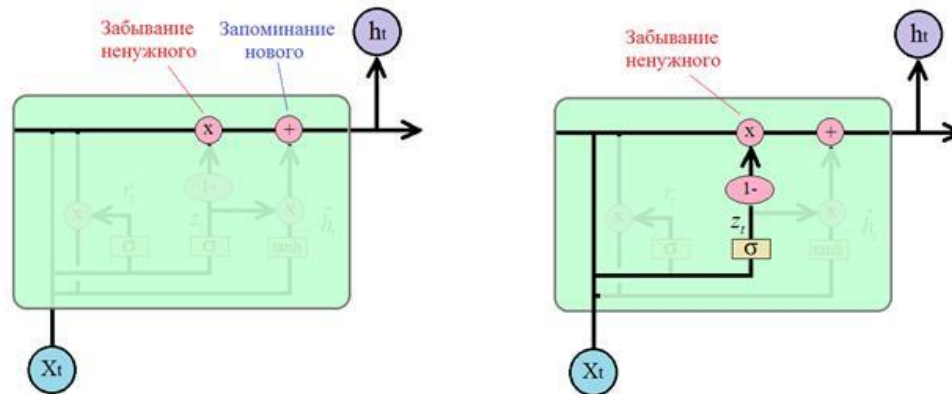
Виды архитектуры RNN. GRU

- Управляемые рекуррентные блоки: Gated Recurrent Units (GRU)
- Вариация RNN, поддерживающая выборочное сохранение памяти. В эту модель добавлены шлюзы обновления и забывания в скрытом слое, что позволяет хранить информацию в памяти и удалять ее



Виды архитектуры RNN. GRU

- Работа этого блока (ячейки) похожа на блок LSTM только здесь долгосрочный элемент памяти реализован не отдельным каналом, а в самом векторе скрытого состояния .
- Две поэлементные операции умножения и сложения для забывания ненужной информации и запоминания новой:



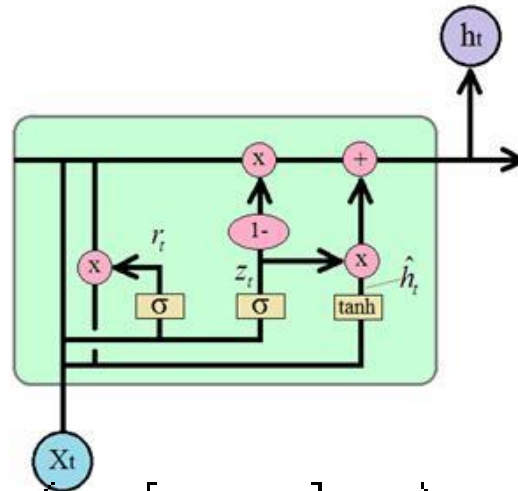
- Для определения что забыть, а что оставить, используется вектор

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

- который, затем, поэлементно вычитается из 1 и умножается на вектор предыдущего состояния

$$h_{t-1} \otimes (1 - z_t)$$

Виды архитектуры RNN. GRU



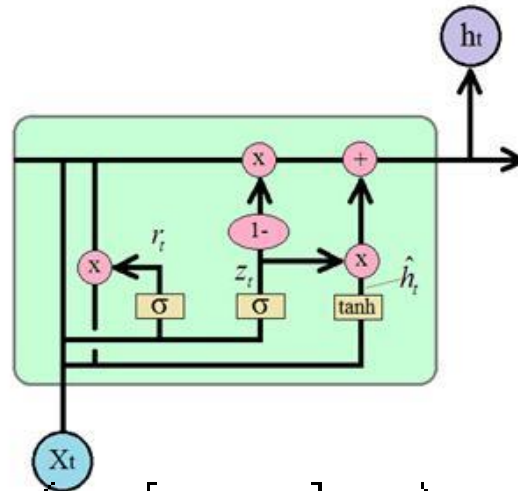
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

$$\hat{h}_t = \tanh(W_{xh} \cdot x_t + W_{rh} \cdot (r_t \otimes h_{t-1}) + b_h)$$

И вычисленная величина добавляется как то, что нужно «запомнить» в векторе скрытого состояния для следующей итерации. В итоге, вычисление имеет вид:

$$h_t = h_{t-1} \otimes (1 - z_t) + \hat{h}_t \otimes z_t$$

Виды архитектуры RNN. GRU



$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

$$\hat{h}_t = \tanh(W_{xh} \cdot x_t + W_{rh} \cdot (r_t \otimes h_{t-1}) + b_h)$$

И вычисленная величина добавляется как то, что нужно «запомнить» в векторе скрытого состояния для следующей итерации. В итоге, вычисление имеет вид:

$$h_t = h_{t-1} \otimes (1 - z_t) + \hat{h}_t \otimes z_t$$

Виды архитектуры RNN. Программная реализация LSTM, GRU

- Используем встроенные возможности Keras(RNN + RNN API)
<https://ru-keras.com/recurrent-layers/>
- В TensorFlow 2.0 есть базовый функционал для построения
https://www.tensorflow.org/api_docs/python/tf/keras/layers/RNN ,
встроенные слои LSTM и GRU пригодны для использования ядер CuDNN по умолчанию, если доступен графический процессор.
<https://habr.com/ru/articles/487808/>

Оптимизаторы в RNN



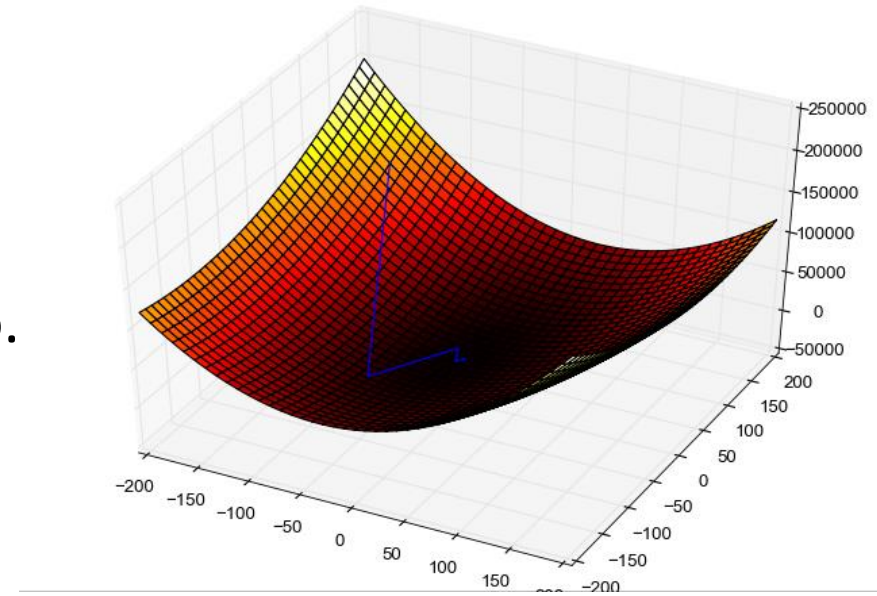
- **Функция потерь** - метрика по которой мы оцениваем модель. Используется для оценки качества модели. Показывает, насколько точно модель предсказывает выходные данные на основе входных данных. Чем меньше значение функции потерь, тем лучше модель подходит для данных.
- **Оптимизатор** - алгоритм подбирающий параметры тела модели опираясь на функцию потерь. Используется для минимизации функции потерь. Определяет, какие изменения необходимо внести в тело модели, чтобы уменьшить значение функции потерь. Различные оптимизаторы имеют различные свойства и могут быть использованы в зависимости от сложности модели и задачи.

Функция потерь (опять!)

- Стохастический градиентный спуск (SGD).
- Оптимизатор импульса (Momentum).
- Среднеквадратичное распространение (RMSProp).
- Адаптивная оценка момента (Adam).

Функция потерь. Стохастический градиентный спуск (SGD)

- Сеть с двумя весовыми параметрами:
 - На горизонтальной плоскости размещаются две оси для весов w_1 и w_2 , соответственно.
 - На вертикальной оси откладываются значения потерь для каждого сочетания весов.
- Ландшафт потерь - кривая представляет потери для различных значений весов при сохранении неизменным фиксированного набора входных данных.



Синяя линия - траектория алгоритма градиентного спуска при оптимизации

Функция потерь. Стохастический градиентный спуск (SGD)

- Работа алгоритма начинается с выбора некоторых случайных значений обоих весов и вычисления значения потери.
- После каждой итерации весовые значения будут обновляться (в результате чего, как можно надеяться, снизится потеря), и алгоритм будет перемещаться вдоль кривой к следующей (более нижней) точке.
- Алгоритм достигает цели, то есть приходит в нижнюю точку кривой — в точку с самым низким значением потерь.
- Градиент измеряет уклон и рассчитывается как значение изменения в вертикальном направлении (dL), поделённое на значение изменения горизонтальном направлении (dW). Другими словами, для крутых уклонов значение градиента большое, для пологих — маленькое.



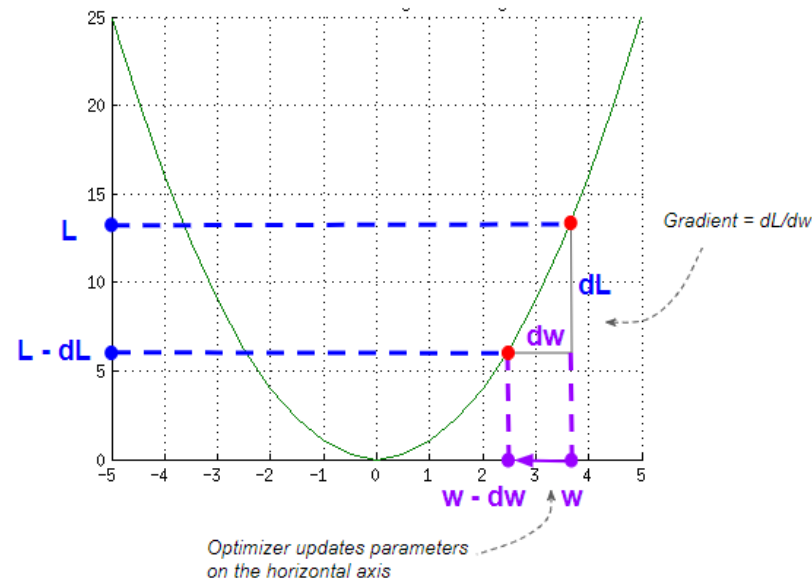
Updated weight

Current weight

$$w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w}$$

Learning Rate

Gradient



Функция потерь. Стохастический градиентный спуск (SGD)

- Обычный градиентный спуск (по умолчанию) "градиентный спуск со сменой коэффициентов после обчёта всей выборки", то есть потери и градиент рассчитываются с использованием всех элементов набора данных
- Стохастический градиентный спуск с мини-пакетами — вариант, при котором коэффициенты меняются после обчёта N элементов выборки, то есть для каждой тренировочной итерации алгоритм выбирает случайное подмножество набора данных.
- Случайность выбора помогает глубже исследовать ландшафт потерь.
- Кривая потерь получается за счёт изменения параметров модели с сохранением фиксированного набора входных данных. Изменяя набор входных данных, для каждого мини-пакета мы получим собственную кривую потерь, которая немного отличается от других
- Если алгоритм застрянет на каком-либо месте ландшафта в одном мини-пакете, можно будет запустить другой мини-пакет с другим ландшафтом, и вполне вероятно, что алгоритм сможет двигаться дальше. Это предотвращает застревание алгоритма на определённых участках ландшафта, особенно на ранних этапах тренировки

Функция потерь. Стохастический градиентный спуск (SGD)

```
def RandomMiniBatches(X, Y, MiniBatchSize):

    m = X.shape[0]
    miniBatches = []

    permutation = list(np.random.permutation(m))
    shuffled_X = X[permutation, :]
    shuffled_Y = Y[permutation, :].reshape((m,1))    #sure for uptpur shape

    num_minibatches = m // MiniBatchSize
    for k in range(0, num_minibatches):
        miniBatch_X = shuffled_X[k * MiniBatchSize:(k + 1) * MiniBatchSize,:]
        miniBatch_Y = shuffled_Y[k * MiniBatchSize:(k + 1) * MiniBatchSize,:]
        miniBatch = (miniBatch_X, miniBatch_Y)
        miniBatches.append(miniBatch)

    #handeling last batch
    if m % MiniBatchSize != 0:
        # end = m - MiniBatchSize * m // MiniBatchSize
        miniBatch_X = shuffled_X[num_minibatches * MiniBatchSize:, :]
        miniBatch_Y = shuffled_Y[num_minibatches * MiniBatchSize:, :]

        miniBatch = (miniBatch_X, miniBatch_Y)
        miniBatches.append(miniBatch)

    return miniBatches
```

https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/experimental/SGD

Функция потерь. Накопление импульса (Momentum)

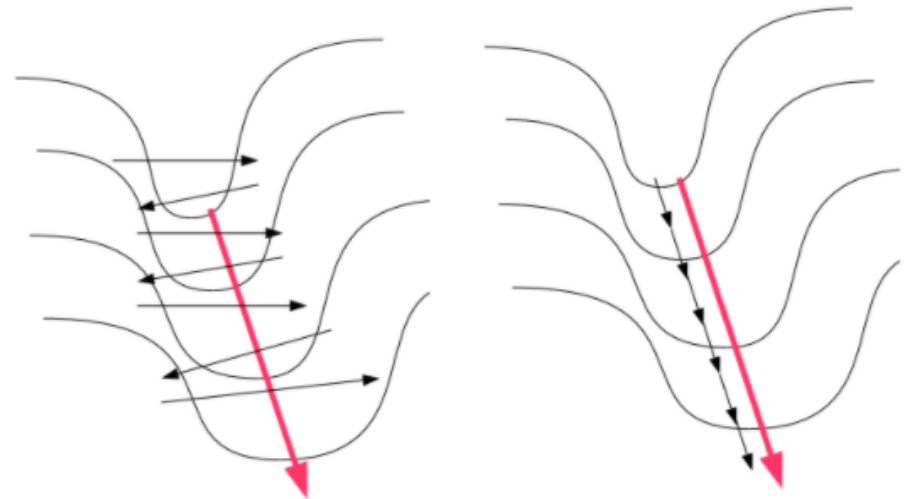
- Поведение алгоритма на крутых склонах: так как градиент на крутых уклонах большой, алгоритм в этих местах может делать большие шаги, хотя требуются маленькие. Алгоритм будет "скакать" назад и вперёд, замедляя тем самым процесс тренировки.
- Выход – динамическое изменение частоты обновлений, чтобы алгоритм мог подстраиваться под изменения окружающего ландшафта. Если уклон очень крутой, алгоритм должен притормаживать. Если склон довольно пологий, скорость вполне можно повысить.
- Алгоритм градиентного спуска обновляет веса на каждом шаге, используя в качестве параметров значения градиента и скорости обучения. Таким образом, для изменения количества обновлений можно выполнить два действия:
 - Скорректировать градиент.
 - Скорректировать значение скорости обучения

Функция потерь. Накопление импульса (Momentum)

- Скорректировать градиент
- Выполняется с помощью функции накопления импульса, учитываются значения прошлых градиентов, общее направление сохраняется, и траектория остаётся правильной.
- Позволяет использовать знания об окружающем ландшафте, полученные алгоритмом до того, как он добрался до данной точки, и смягчить эффект аномалии кривой потерь.
- Использует при работе экспоненциальное скользящее среднее, а не текущее значение градиента

Где брать код и на SGD и Momentum?

<https://keras.io/api/optimizers/sgd>



SGD bounces back and forth from one side of the valley to the other

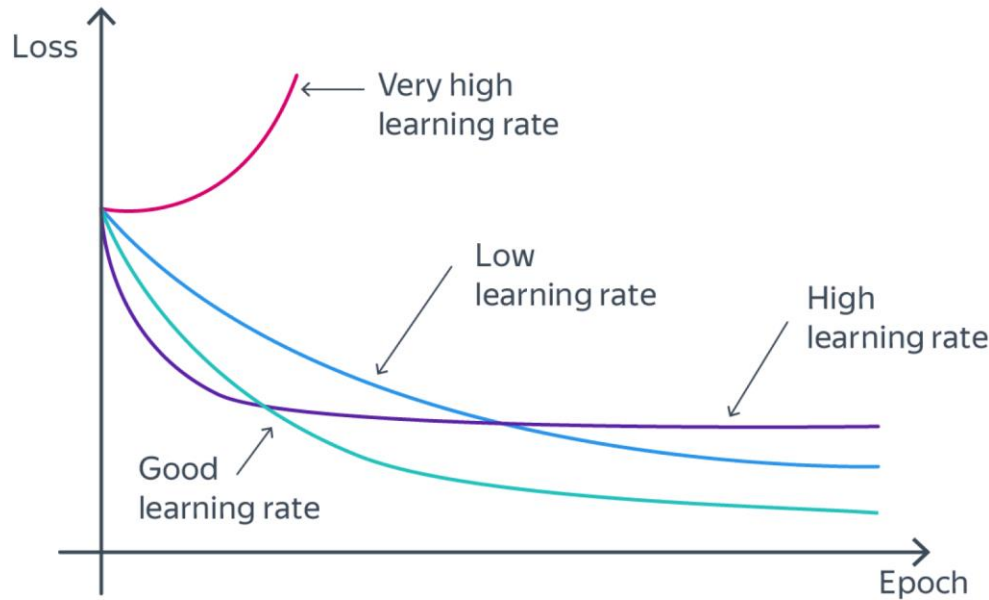
Using Momentum the zig-zag cancels out, while the direction along the valley is reinforced

Функция потерь. Скорректировать значение скорости обучения (RMSProp)

- Возводит в квадрат значения прошлых градиентов, но приводит их к экспоненциальному скользящему среднему, тем самым придавая более высокую значимость последним по времени градиентам
- После возведения градиентов в квадрат все они становятся положительными, то есть получают одно и то же направление => перестаёт действовать эффект, проявляющийся при применении функции накопления импульса (градиенты имеют противоположные направления).
- Для параметра с крутым уклоном значения градиентов будут большими, и возведение таких значений в квадрат даст ещё большее и всегда положительное значение, поэтому такие градиенты быстро накапливаются.
- Для погашения данного эффекта алгоритм рассчитывает скорость обучения посредством деления накопленных квадратов градиентов на коэффициент с большим значением. За счет этого алгоритм “притормаживает” на крутых уклонах.
- Если уклоны небольшие, накопления также будут небольшими, поэтому при вычислении скорости обучения алгоритм разделит накопленные квадраты градиентов на коэффициент с меньшим значением. Таким образом, скорость обучения на пологих склонах будет увеличена.

Функция потерь. Скорректировать значение скорости обучения (RMSProp)

- Благодаря адаптивному подбору шага не нужно подбирать последовательность размеров всех шагов, а достаточно выбрать всего одно число – learning rate , всё остальное сделает сам алгоритм.
- Learning rate нужно выбирать крайне аккуратно: алгоритм может либо преждевременно выйти на плато, либо вовсе разойтись. Пример приведен на иллюстрации ниже.



- https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/experimental/RMSprop

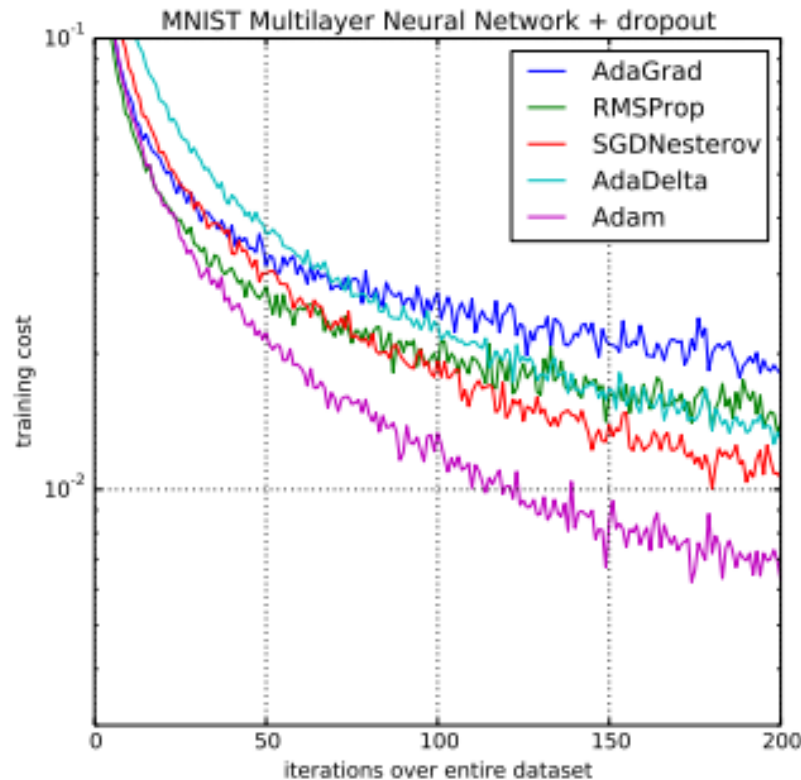
Функция потерь. Adam

- [Adam](#) — adaptive moment estimation, ADaptive Momentum сочетает в себе и идею накопления движения и идею более слабого обновления весов для типичных признаков ([статья](#))
- Метод вычисляет индивидуальные адаптивные скорости обучения для различных параметров из оценок первого и второго моментов градиентов.
- Включает в себя:
 - Адаптивный Градиентный Алгоритм, который поддерживает скорость обучения по параметру, которая улучшает производительность при проблемах с разреженными градиентами (например, проблемы с естественным языком и компьютерным зрением).
 - RMSProp, который поддерживает скорости обучения по каждому параметру, которые адаптированы на основе среднего значения последних величин градиентов для веса (например, как быстро оно изменяется). Это означает, что алгоритм хорошо справляется с онлайн и нестационарными задачами (например, с шумом).

Функция потерь. Adam

- **Параметры конфигурации Адама**
 - **альфа**, Также называется скоростью обучения или размером шага. Пропорция веса обновляется (например, 0,001). Большие значения (например, 0,3) приводят к более быстрому начальному обучению до обновления скорости. Меньшие значения (например, 1.0E-5) замедляют обучение прямо во время тренировки
 - **бета1**, Показатель экспоненциальной скорости затухания для первого момента (например, 0,9).
 - **бета2**, Показатель экспоненциального спада для оценок второго момента (например, 0,999). Это значение должно быть близко к 1,0 в случае проблем с разреженным градиентом (например, проблемы NLP и компьютерного зрения).
 - **эпсилон**, Это очень небольшое число, чтобы предотвратить любое деление на ноль при реализации (например, 10E-8).

Функция потерь. Adam



- Адам использует среднее значение вторых моментов градиентов (нецентрированная дисперсия).
- Алгоритм вычисляет экспоненциальную скользящую среднюю градиента и квадрата градиента, а параметры бета1 и бета2 управляют скоростью затухания этих скользящих средних.
- Начальное значение скользящих средних и значений бета1 и бета2, близких к 1,0 (рекомендуется), приводит к смещению оценок моментов в сторону нуля. Это смещение преодолевается сначала вычислением смещенных оценок, а затем вычислением оценок с поправкой на смещение.
- Код <https://keras.io/api/optimizers/adam/>
- https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/Adam

Регуляризация

- Регуляризация метод добавления некоторых дополнительных ограничений к условию с целью решить некорректно поставленную задачу или **предотвратить переобучение**.
- Регуляризация включает в себя добавление штрафов к функции потерь, которые препятствуют использованию больших весов или активаций в модели.
- Методы регуляризации для предотвращения переобучения в моделях:
 - L1 - Lasso (Least Absolute Shrinkage and Selection Operator) регуляризация. Основана на добавлении штрафа, равного абсолютному значению коэффициентов модели.
 - L2 - Ridge регуляризация - применяется в линейной регрессии и многих других моделях и добавляет к оптимизационной функции модели штрафную функцию

Регуляризация L1

- L1 регуляризация добавляет в функцию потерь дополнительное слагаемое налагающее штраф за сложность модели, то есть высокие веса:

L1 регуляризация

$$L_1 = \underbrace{\sum_i (y_i - y(t_i))^2}_{\text{ошибка модели по метрике}} + \lambda \underbrace{\sum_i |a_i|}_{\text{штраф}}$$

- При использовании метода градиентного спуска, L1 регуляризация может привести к некоторым проблемам:
- Имеет несколько «острых» углов (разрывов) в окрестности нуля, где производная не определена.
- Это усложняет вычисление градиента функции потерь, когда используется L1 регуляризация.
- Метод градиентного спуска требует, чтобы градиент был гладким и непрерывным, чтобы правильно работать, и поэтому L1 регуляризация может быть менее эффективна при использовании градиентного спуска.

Регуляризация L2

- Вместо L1 регуляризации в методе градиентного спуска часто используется L2 регуляризация, так как она имеет более гладкую производную и может лучше работать с градиентным спуском.
- L2 регуляризация добавляет к оптимизационной функции модели штрафную функцию - сумму квадратов весов модели, умноженных на гиперпараметр регуляризации

L2 регуляризация

$$L_2 = \sum_i (y_i - y(t_i))^2 + \lambda \sum_i a_i^2$$

ошибка модели
по метрике

штраф

- L2 регуляризация штрафует, не обнуляя значения весов, заставляя их приближаться к нулю, но в отличие от L1 регуляризации не зануляет их полностью. Вместо этого L2 регуляризация штрафует большие значения весов более гладко и непрерывно, что позволяет более уверенно управлять компромиссом между точностью и сложностью модели.
- L2 регуляризация может помочь в предотвращении переобучения и улучшении обобщающей способности модели, а также в уменьшении влияния шума в данных на модель.

Регуляризация в коде

- [Tensowflow L1](#)
- [Tensoeflow L2](#)
- [Tensorflow L1L2](#)
- [Kaggle L1L2](#)

Что делать по лабораторной? (Задача 3)

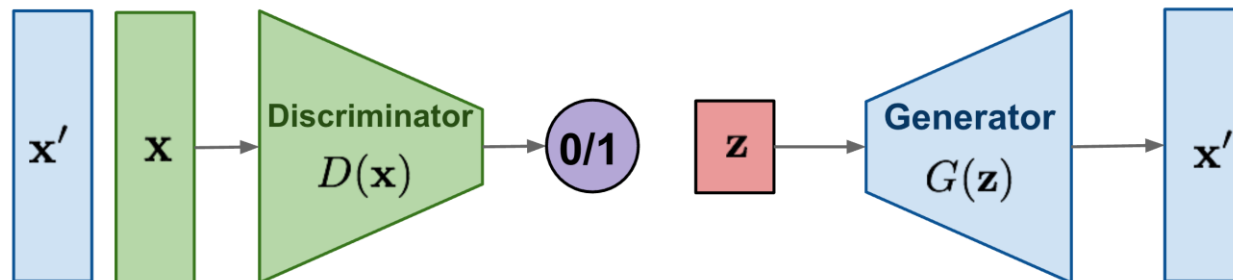
- Применяем на всех 3 архитектурах функции потерь
 - Стохастический градиентный спуск (SGD).
 - Оптимизатор импульса (Momentum).
 - Среднеквадратичное распространение (RMSProp).
 - Адаптивная оценка момента (Adam)
- Применяем на всех 3 архитектурах регуляризацию
 - L1
 - L2

Лабораторная работа 3

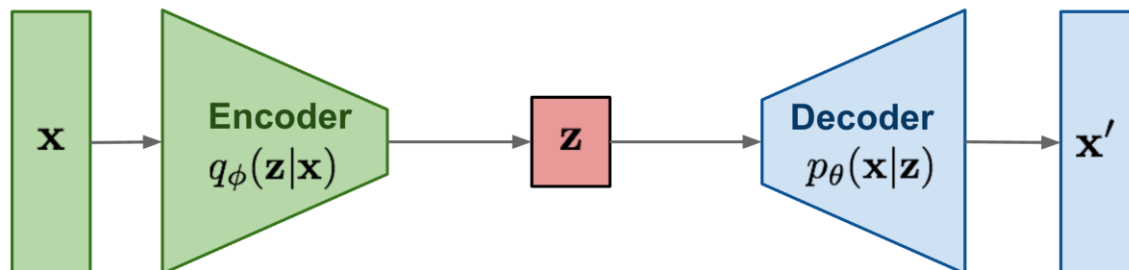
Генеративные нейронные сети

Генеративные нейронные сети

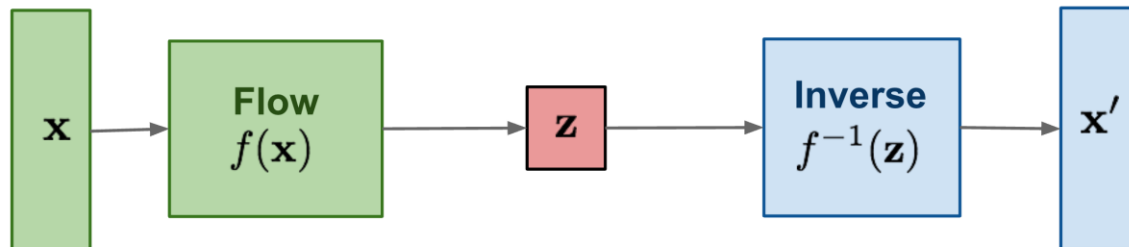
GAN: Adversarial training



VAE: maximize variational lower bound



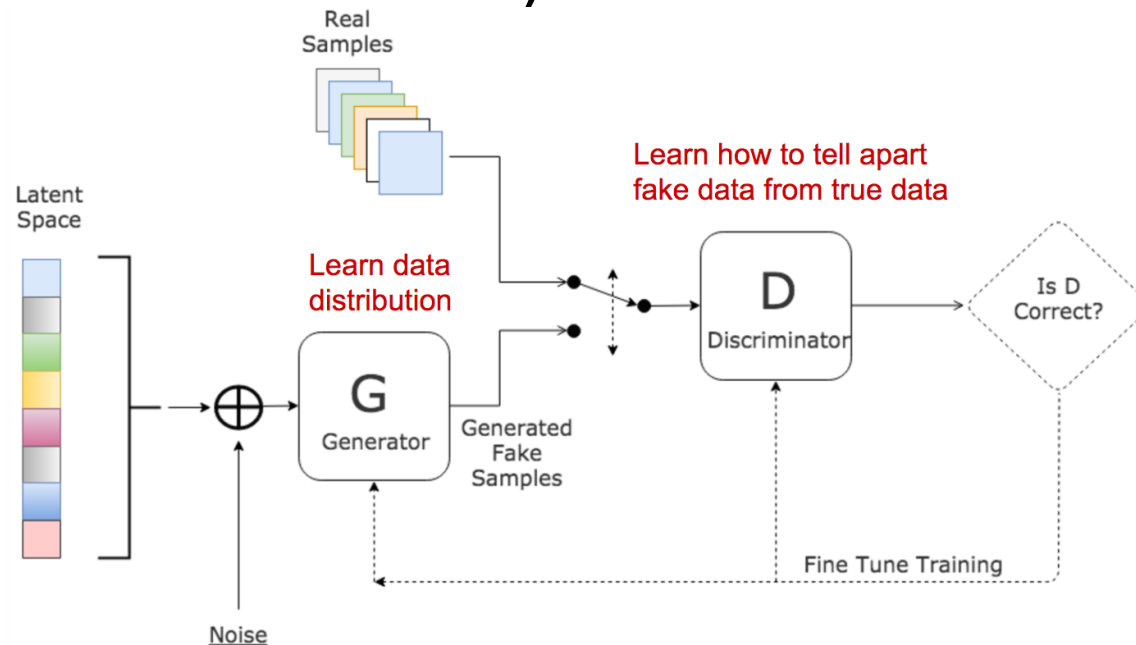
Flow-based models:
Invertible transform of distributions



Diffusion models:
Gradually add Gaussian noise and then reverse



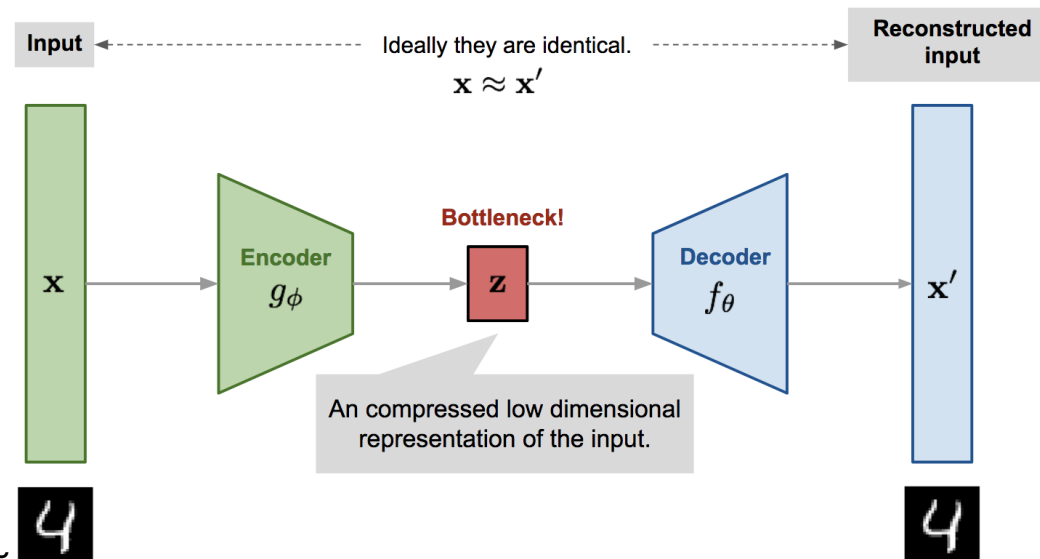
Генеративные модели. GAN - (Generative Adversarial Networks)



Дискриминатор D оценивает вероятность отнесения образцов к настоящим или нет, используя реальный датасет. Генератор G производит ситретические изображения с добавлением шума, таким образом, чтобы Дискриминатор их отнес к реальным. Таким образом они соревнуются, насколько Генератор сможет обмануть Дискриминатор.

Хорошая статья <https://lilianweng.github.io/posts/2017-08-20-gan/>

Генеративные модели. VAE (Variational Autoencoders).



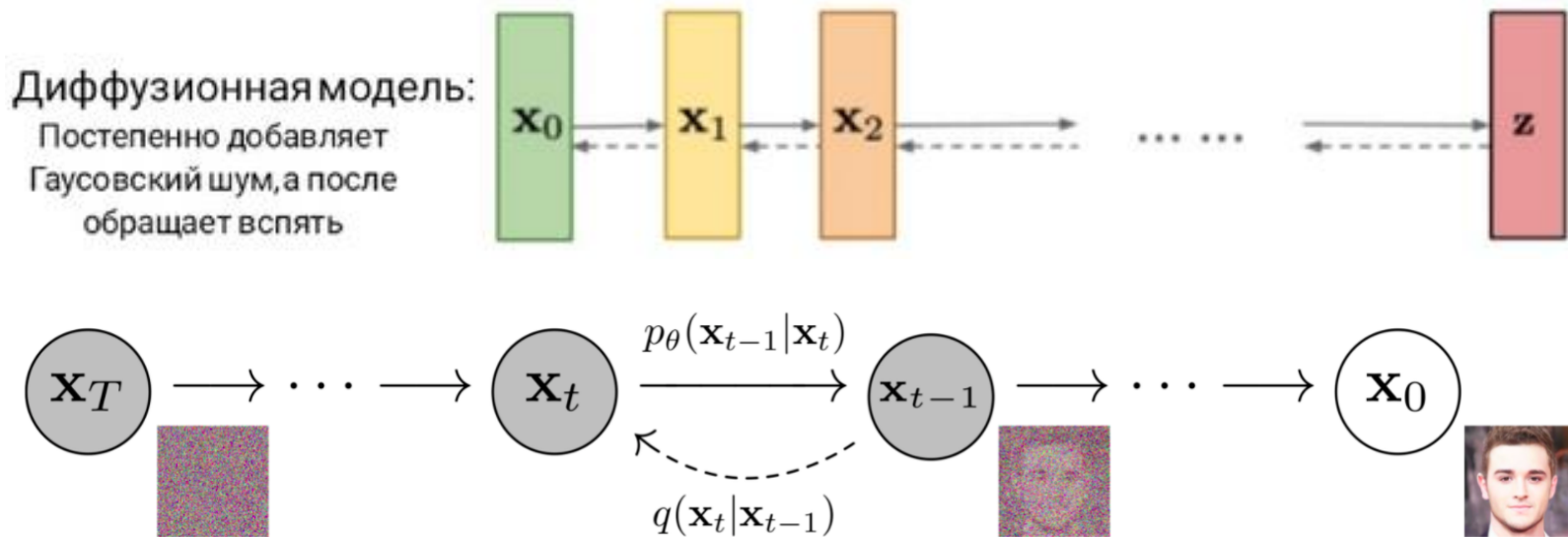
Состоит из двух сетей.

- Энкодер – переводит изображение высокого разрешения в код низкого разрешения (свертка)
- Декодер – извлекает данные из кода, добавляя шумы с использованием байесовской и графовой моделей.

VAE переводит входное изображение не в фиксированный вектор, а в распределенные вектора, строя связи между входной датой и векторами на основе параметров:

- Prior - приоритетность признака
- Likelihood схожесть признака
- Posterior - Схожесть сгенерированного изображения с первичным

Генеративные модели. Диффузионная модель



Diffusion probabilistic models are parameterized Markov chains trained to gradually denoise data. We estimate parameters of the generative process p .

- Берётся начальное изображение
- Итеративно добавляется Гауссовский шум, пока от исходного ничего не останется, кроме пикселей
- Добавляется модель шумоподавления, которая обращает процесс так, чтобы привести его к результату, похожему на исходное изображение

Генеративные модели. Диффузионная модель

Импорт библиотек

```
!pip install datasets &>> install.log
```

```
from datasets import load_dataset
```

```
from PIL import Image
```

```
import torch.nn.functional as F
```

```
import os
```

```
from tqdm.notebook import tqdm
```

```
import torch
```

```
import numpy as np
```

```
def img_to_tensor(im):
```

```
    return torch.tensor(np.array(im.convert('RGB'))/255).permute(2, 0, 1).unsqueeze(0) * 2 - 1
```

```
def tensor_to_image(t):
```

```
    return Image.fromarray(np.array((((t.squeeze()).permute(1, 2, 0)+1)/2).clip(0, 1)*255).astype(np.uint8))
```

```
def gather(consts: torch.Tensor, t: torch.Tensor):
```

```
    c = consts.gather(-1, t)
```

```
    return c.reshape(-1, 1, 1, 1)
```

Генеративные модели. Диффузионная модель

Импортируем библиотеки и изображения (32*32), можно использовать изображения из лабораторной 1

```
!pip install datasets &>> install.log
from datasets import load_dataset
from PIL import Image
import torch.nn.functional as F
import os
from tqdm.notebook import tqdm
import torch
import numpy as np
def img_to_tensor(im):
    return torch.tensor(np.array(im.convert('RGB'))/255).permute(2, 0, 1).unsqueeze(0) * 2 - 1
def tensor_to_image(t):
    return Image.fromarray(np.array((((t.squeeze()).permute(1, 2, 0)+1)/2).clip(0, 1)*255).astype(np.uint8))

def gather(consts: torch.Tensor, t: torch.Tensor):
    c = consts.gather(-1, t)
    return c.reshape(-1, 1, 1, 1)
# Скачать и загрузить набор данных
cifar10 = load_dataset('cifar10')
image = Image.new('RGB', size=(32*5, 32*2))
for i in range(10):
    im = cifar10['train'][i]['img']
    image.paste(im, ( (i%5)*32, (i//5)*32 ))
    image.resize((32*5*4, 32*2*4), Image.NEAREST)
```

Генеративные модели. Диффузионная модель

Добавляем шум с использованием [репараметризации](#)

```
n_steps = 100
beta = torch.linspace(0.0001, 0.04, n_steps)
alpha = 1. - beta
alpha_bar = torch.cumprod(alpha, dim=0)
def q_xt_x0(x0, t):
    mean = gather(alpha_bar, t) ** 0.5 * x0 # теперь alpha_bar
    var = 1-gather(alpha_bar, t) # (1-alpha_bar)
    eps = torch.randn_like(x0)
    return mean + (var ** 0.5) * eps
```

Показать на разных этапах

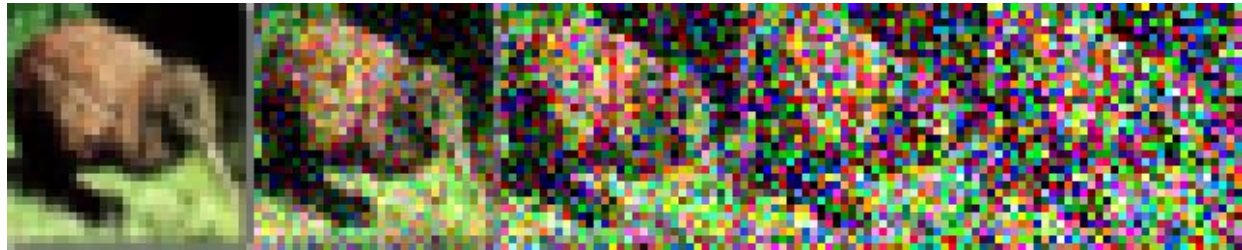
```
ims = []
start_im = cifar10['train'][3]['img']
x0 = img_to_tensor(start_im).squeeze()
for t in [0, 20, 40, 60, 80]:
    x = q_xt_x0(x0, torch.tensor(t, dtype=torch.long))
    ims.append(tensor_to_image(x))
```

```
image = Image.new('RGB', size=(32*5, 32))
```

```
for i, im in enumerate(ims):
```

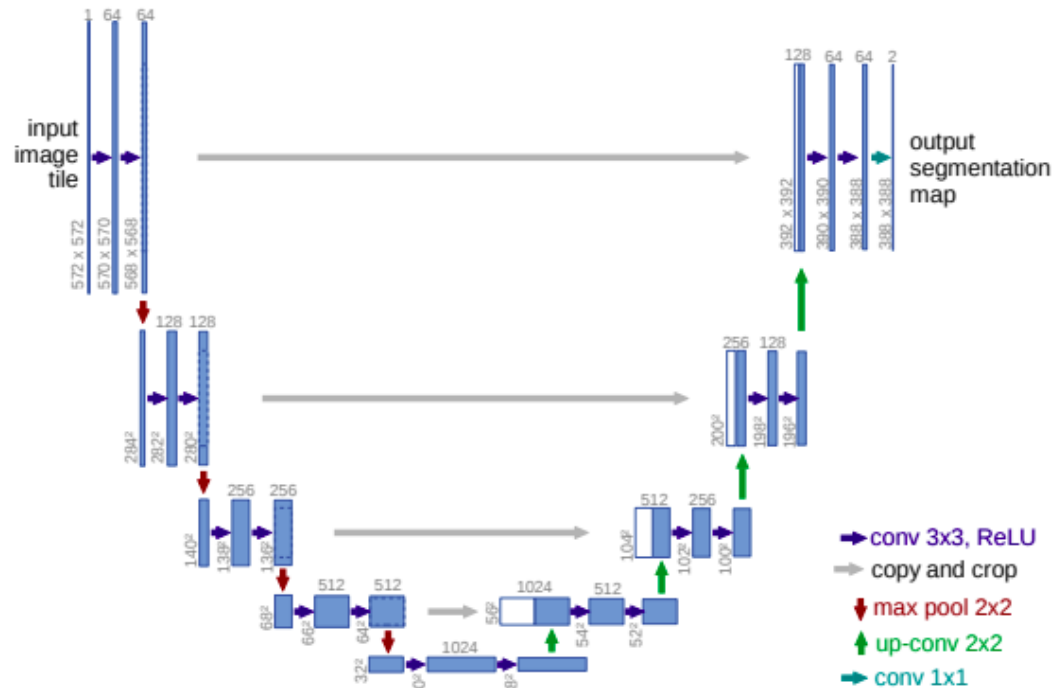
```
    image.paste(im, ((i%5)*32, 0))
```

```
image.resize((32*4*5, 32*4), Image.NEAREST)
```



Генеративные модели. Диффузионная модель

Добавляем шумоподавление с помощью U-net



- Архитектура передаёт информацию с ранних уровней с высоким разрешением — в более поздние уровни с использованием ярлыка, что позволяет использовать подробные функции исходного изображения, а также захватывать более высокоуровневую семантику из глубоких слоёв.
- Диффузионные нейросети используют архитектуру U-NET с одним дополнением: TimeEmbedding, который кодирует временной шаг (t) и позволяет модели использовать это в качестве информации для кондиционирования, передавая её в середине сети.
- [Код U-net](#)

Генеративные модели. Диффузионная модель

Обучение модели нам нужно предсказывать шум с учётом x_t и t .
Настраиваем функцию $q_{x_t|x_0}$ так, чтобы она возвращала как изображение с шумом (x_t), так и сам шум, который будет целью, которую пытается создать наша модель.

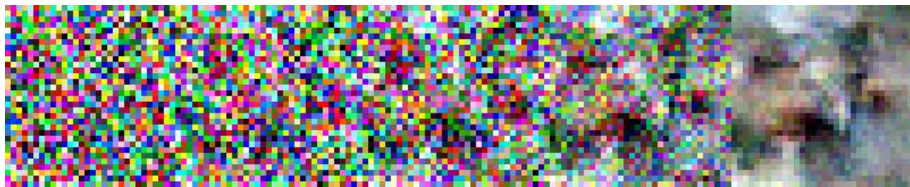
[Вот тут ссылка на код](#)

Шумоподавление

Определяем шумоподавление $p_{\theta}(x_{t-1}|x_t)$.

Мы используем нашу модель для прогнозирования шума, а затем «отменяем» шаги прямого шума по одному за раз, чтобы перейти от изображения, представляющего собой чистый шум, к изображению, которое выглядит похоже на реальное изображение:

[Ссылка на код](#) и [на код с лучшим шумоподавлением](#)



Генеративные модели. Диффузионная модель

[Колаб с объяснением и тьюториалом](#)